

ระบบเทรดทองคำอัตโนมัติ

xAuby

คู่มือฉบับสมบูรณ์ — สถาปัตยกรรม การตั้งค่า กลยุทธ์ Market
Regime Detector การจัดการความเสี่ยง และการดูแลระบบ

รวบรวมและเรียบเรียงจากเอกสารทั้งหมดของโปรเจกต์
(README, README_DEV, CLAUDE.md, DESIGN.md,
PRODUCT.md, docs/*)

อ้างอิงสถานะระบบล่าสุด ณ กรกฎาคม 2026
เอกสารนี้จัดทำเพื่อการศึกษาและใช้งานภายในเท่านั้น

สารบัญ

คำนำ และสถานะระบบล่าสุด

ภาคที่ 1 — รู้จัก xAuby

- 1.1 xAuby คืออะไร
- 1.2 ปรัชญาการออกแบบ
- 1.3 สถานะการทำงานปัจจุบัน (Current Runtime Baseline)
- 1.4 สิ่งที่ทำและไม่ทำ

ภาคที่ 2 — เริ่มต้นใช้งาน

- 2.1 การติดตั้ง
- 2.2 การตั้งค่า API Key ของ OKX
- 2.3 รับโหมดจำลองก่อนเสมอ
- 2.4 คำสั่งใช้งานที่พบบ่อย
- 2.5 เช็กलिस्टก่อนเปิดใช้งานจริง (Live)

ภาคที่ 3 — สถาปัตยกรรมระบบ

- 3.1 ภาพรวมโครงสร้างเลเยอร์
- 3.2 หนึ่ง Tick ของเอนจินทำอะไรบ้าง
- 3.3 PairRegistry และ SymbolContext
- 3.4 กฎการพึ่งพาของระบบ (Dependency Rules)
- 3.5 ปลั๊กอิน Strategy และ Indicator
- 3.6 Feature Flags ทั้งหมด (architecture.*)

ภาคที่ 4 — การตั้งค่าระบบ (Configuration)

- 4.1 ไฟล์ตั้งค่า 3 ไฟล์หลัก
- 4.2 หลักการ Source of Truth 3 ชั้น
- 4.3 การตั้งค่า Exchange (OKX ผ่าน CCXT)
- 4.4 โครงสร้าง Whitelist และคู่เทรดปัจจุบัน
- 4.5 ตัวแปรแวดล้อม (Environment Variables)
- 4.6 เมนู Quick Config ในตัว

ภาคที่ 5 — กลยุทธ์และ Market Regime Detector

- 5.1 กลยุทธ์ (Strategy Plugin) ทั้งหมดในระบบ
- 5.2 CDC Action Zone — กลยุทธ์หลักที่ใช้งานจริง
- 5.3 Market Regime Detector คืออะไร ทำงานอย่างไร
- 5.4 ตัวจับ Regime เชิงสถิติ (GMM Cross-check)

- 5.5 รายงานความแม่นยำของ Regime Detector
- 5.6 RegimeRouter — เลือกกลยุทธ์ตาม Regime อัตโนมัติ
- 5.7 กระบวนการคัดเลือกกลยุทธ์ด้วยข้อมูลจริง (R&D)

ภาคที่ 6 — การจัดการความเสี่ยง

- 6.1 การคำนวณขนาดโพสิชัน (Position Sizing)
- 6.2 Stop-Loss / Trailing / Breakeven
- 6.3 Partial Take-Profit
- 6.4 Drawdown Guard — สวิตช์ตัดขาดทุนระดับพอร์ต
- 6.5 การเทรด Short และเงื่อนไขความปลอดภัย
- 6.6 เช็กลิสต์ความสอดคล้องของค่าความเสี่ยง

ภาคที่ 7 — Backtest และงานวิจัยกลยุทธ์

- 7.1 แหล่งข้อมูลสำหรับ Backtest
- 7.2 คำสั่ง Backtest และ Optimizer
- 7.3 Regime-aware Replay
- 7.4 Replay Validation หลัง Restart/Incident

ภาคที่ 8 — หน้าจอใช้งาน (TUI / WebUI)

- 8.1 Textual TUI (แดชบอร์ดในเทอร์มินัล)
- 8.2 WebUI มือถือ (Read-only)
- 8.3 ปรัชญาการออกแบบหน้าจอ

ภาคที่ 9 — Telegram

- 9.1 การติดตั้ง Telegram Bot
- 9.2 คำสั่งทั้งหมด
- 9.3 การแจ้งเตือนอัตโนมัติ
- 9.4 โหมด Semi-auto และ Emergency Pause

ภาคที่ 10 — หลายบัญชี และหลาย Exchange

- 10.1 รับหลายอินสแตนซ์บนเครื่องเดียว
- 10.2 Multi-Exchange ผ่าน CCXT
- 10.3 OKX Perpetual และความปลอดภัยของ Short

ภาคที่ 11 — การดูแลระบบ (Operations)

- 11.1 การ Deploy บน VPS
- 11.2 คำสั่งอัปเดตและรีสตาร์ท
- 11.3 การตรวจสอบ Latency ของ VPS
- 11.4 เช็กลิสต์ความปลอดภัยฉบับเต็ม

ภาคผนวก

- ก. อภิธานศัพท์ (Glossary)
- ข. โครงสร้างโปรเจกต์

- ค. ตารางสคริปต์อ้างอิงทั้งหมด
- ง. คำเตือนความเสี่ยง (Disclaimer)

คำนำ และสถานะระบบล่าสุด

หนังสือเล่มนี้รวบรวมเอกสารทั้งหมดของโปรเจกต์ xAuby — `README.md`, `README_DEV.md`, `CLAUDE.md`, `DESIGN.md`, `PRODUCT.md` และเอกสารทั้งหมดใน `docs/` — มาจัดเรียงใหม่เป็นลำดับการอ่านเดียว แปลและเรียบเรียงเป็นภาษาไทยที่อ่านง่าย ตัดส่วนที่ซ้ำซ้อนออก และแก้ไขจุดที่เอกสารต้นฉบับลำสมัย (doc drift) ให้ตรงกับสถานะจริงของระบบ ที่อ่านได้จากไฟล์ config จริง (`bot_config.yaml`, `coin_whitelist.json`)

หมายเหตุสำคัญเรื่องความคลาดเคลื่อนของเอกสารเดิม: เอกสารต้นฉบับหลายไฟล์ (โดยเฉพาะ `README_DEV.md`, `docs/architecture.md`, `docs/trading-flow.md`, `docs/configuration.md`, `docs/tui.md`, `docs/webui.md`, `docs/telegram.md`) ยังอธิบายสถานะเก่าของระบบ (Binance.th แบบ Spot, คู่เทรด `XAUTUSDT` + `BTCUSDT` สองคู่พร้อมกัน, `risk_pct 0.45`) ซึ่ง `CLAUDE.md` เองก็ระบุไว้ตรงๆ ว่าเป็น "doc drift warning" หนังสือเล่มนี้ยึด `coin_whitelist.json` และ `bot_config.yaml` **จริงเป็นความจริงหลัก (ground truth)** ตามที่ `CLAUDE.md` กำหนดไว้ และจะระบุชัดเจนทุกจุดที่เนื้อหาอธิบาย "ความสามารถของระบบ" (เช่น รองรับหลายคู่เทรด, รองรับหลาย exchange) แยกจาก "สิ่งที่กำลังทำงานจริงตอนนี้" (คู่เทรดเดียวคือทองคำบน OKX)

สถานะการทำงานจริงของระบบ ณ ปัจจุบัน (สรุปสั้น)

รายการ	ค่าจริงในระบบ
Exchange	OKX — USDT-settled Perpetual Swap ผ่าน CCXT adapter
คู่เทรดที่ Live	<code>XAUUSDT</code> (ทองคำ) เพียงคู่เดียว
กลยุทธ์	<code>cdc_action_zone</code> แบบ Long + Short (stop-and-reverse)
Timeframe	หลัก 4 ชั่วโมง (4h) / ยืนยัน 1 วัน (1d)
Leverage	1 เท่า (isolated margin, one-way position)
ความเสี่ยงต่อไม้	<code>risk_pct = 0.02</code> (2% ของ Equity)
Max allocation ต่อไม้	25%
Max daily loss	6%
Max open positions	1
Stop-Loss	ปิดใช้งาน (<code>disable_stop_loss: true</code>) — เป็นกลยุทธ์ CDC-pure ออกด้วยการกลับโซนเท่านั้น
Partial Take-Profit	ปิด 50% ของโพสิชันเมื่อกำไร +12%
RegimeRouter	ปิดอยู่สำหรับคู่ XAU (<code>regime_router_enabled: false</code>)

ตัวเลขเหล่านี้เปลี่ยนแปลงได้ตามการตั้งค่าจริงของผู้ดูแลระบบ — ให้ตรวจสอบ `bot_config.yaml` และ `coin_whitelist.json` เสมอก่อนอ้างอิงในการตัดสินใจจริง

1.1 xAuby คืออะไร

xAuby คือระบบเทรดอัตโนมัติสำหรับสัญญา Perpetual Swap แบบ single-process รับได้หลายคู่เทรดพร้อมกัน (multi-symbol) บนตลาด **OKX** (api.okx.com) เชื่อมต่อผ่านชั้นกลาง (adapter) ที่เป็นกลางกับ exchange ชื่อ **CCXT** ระบบดึงราคาแท่งเทียนและราคาล่าสุด ผ่านทั้ง REST API และ WebSocket, รับแต่ละคู่เทรดผ่านปลั๊กอินกลยุทธ์ของตัวเอง, สามารถส่งผ่านตัวจำแนกสภาพตลาด (Market Regime Classifier) เพื่อเลือกกลยุทธ์อัตโนมัติได้, วางออเดอร์ Swap จริงหรือจำลอง พร้อมระบบ Stop-Loss แบบ ATR / Trailing / Breakeven แล้วบันทึกทุกอย่างลง SQLite และไฟล์ JSONL เพื่อใช้ Replay ตรวจสอบเหตุการณ์ (Incident) และแสดงผลบน Textual Dashboard ผู้ดูแลระบบควบคุมผ่าน CLI, Textual TUI, และ Telegram ชื่อแพ็คเกจ Python คือ **xauby** รองรับ Python ตั้งแต่ 3.10 ขึ้นไป (README กำหนดเป้าหมายที่ 3.12+ ส่วน container พัฒนาปัจจุบันรันบน 3.11)

1.2 ปรัชญาการออกแบบ

หลักการออกแบบที่ยึดถือมาตลอดทั้งระบบ:

- **Config ขับเคลื่อนพฤติกรรม** — เอนจินไม่ hard-code รายชื่อคู่เทรด, กลยุทธ์, หรือเส้นกราฟใดๆ ทั้งสิ้น ทุกอย่างมาจากไฟล์ตั้งค่า
- **เอนจินไม่รู้เรื่องกลยุทธ์ (Strategy-agnostic)** — ตรรกะสัญญาซื้อขายและจุดออกทั้งหมดอยู่ในปลั๊กอิน กลยุทธ์ ไม่ใช่ในเอนจินหลัก การเพิ่มกลยุทธ์ใหม่ไม่ต้องแก้ไขโค้ดเอนจิน
- **แยกความรับผิดชอบ Config เป็น 3 ชั้น** — Engine / Strategy / Portfolio (อธิบายละเอียดในภาคที่ 4) เพื่อไม่ให้ Backtest กับ Live ให้ผลต่างกัน (parity)
- **ปลอดภัยไว้ก่อน (Fail-safe by default)** — ค่าเริ่มต้นเป็นโหมดจำลองเสมอ, การเทรด Live ต้องผ่าน "ประตู" หลายชั้นพร้อมกัน (ดูหัวข้อ 2.5), Router ที่ยังไม่ผ่านการยืนยันจะถูกบังคับกลับเป็น Sim อัตโนมัติ
- **Isolation ต่อคู่เทรด** — แต่ละคู่เทรดที่ Active มี Strategy instance, Runner, โหมด, และ สถานะ handoff ของตัวเอง แยกจากคู่อื่นโดยสมบูรณ์ ส่วนเงินทุนพอร์ตยังคงใช้ร่วมกันโดยเจตนา
- **Rollback คือการปิดสวิตช์ ไม่ใช่ revert โค้ด** — ฟีเจอร์ใหญ่ๆ (RegimeRouter, Event Bus, GMM Cross-check ฯลฯ) ทุกตัวอยู่หลัง feature flag ใน `architecture:` block พลิกกลับเป็น `false` ได้ทันทีโดยไม่ต้องแก้ไขโค้ด

1.3 สถานะการทำงานปัจจุบัน (Current Runtime Baseline)

ตารางด้านล่างคือค่าจริงที่อ่านจาก `coin_whitelist.json` และ `bot_config.yaml` ของ repository ในปัจจุบัน:

คู่เทรด	โหมด	กลยุทธ์	Primary TF	Confirm TF	ฝั่งที่เปิดได้
<code>XAUUSD</code>	live	<code>cdc_action_zone</code>	4h	1d	Long + Short

เอนจินรับ exchange เป็น **OKX Perpetual Swap** (`exchange.provider: ccxt`, `ccxt_id: okx`, `market_type: swap`, `margin_mode: isolated`, `position_mode: one_way`) ที่ **Leverage 1 เท่า** (`derivatives.max_leverage: 1`) พร้อม funding guard (`max_abs_funding_rate`) และ `min_liquidation_distance_pct` ทำงานอยู่ — **1** คือค่าที่ตั้งใช้

งานจริงตอนนี้สำหรับคู่ XAU (ไม่ใช่เพดานตายตัวของ ระบบ) ระบบรองรับเลเวอเรจสูงกว่านี้ได้ทางเทคนิคสำหรับฝั่ง Short โดยเฉพาะ (เพดานสูงสุด 3 เท่า ดูหัวข้อ 6.5) แต่ต้องมีการปรับ `derivatives.max_leverage` และอนุมัติโดยผู้ดูแลระบบก่อนเสมอ ช่องทองใช้สัญลักษณ์ `XAUUSD` (Perpetual) ส่วนงาน Backtest ใช้ `PAXGUSD` เป็นตัวแทนราคาทอง (proxy) เพราะมีประวัติราคาย้อนหลังยาวกว่าบน Global Binance

คู่เทรดทองคำตัวเดียวที่ Live รับกลยุทธ์ CDC Action Zone แบบ **Long และ Short พร้อมกัน** ในลักษณะ stop-and-reverse (`allowed_sides: [long, short]` , `short_live_enabled: true`) เพราะ adapter ของ Swap ประกาศความสามารถ `swap / positions / reduce_only` คสอ (`xauby/api/ccxt_client.py`)

รูปแบบการส่งออเดอร์: การเข้าไม้ใช้ออเดอร์ **LIMIT** ที่ราคาต่ำสุด (`execution.order_type: limit`) และเมื่อ `execution.entry_market_fallback: true` ส่วนที่ยังไม่ Fill จะถูกเติมด้วยออเดอร์ **MARKET** หลังหมดเวลารอ (แทนที่จะยกเลิก) เพื่อให้ Live ตรงกับผล Backtest ที่จำลองการ Fill แบบ Market ส่วนการออกไม้ใช้ MARKET เมื่อเป็นสถานการณ์เร่งด่วน (CDC แดง / NO_TRADE / บังคับปิด) และใช้ LIMIT ในกรณีอื่น

คู่ทองปัจจุบันเป็นแบบ **CDC-pure** (`disable_stop_loss: true`) จึงไม่มี Stop-Loss ฝั่ง Exchange — การออกจากโพซิชันขับเคลื่อนด้วยการกลับโซนเท่านั้น (zone-flip) และการคำนวณขนาดไม้ ใช้ `position_pct` คงที่ของ Equity แทนระยะห่างจาก SL นอกจากนี้ CDC ยังเปิด Partial Take-Profit แบบ one-shot: `partial_tp_pct: 12.0` , `partial_tp_fraction: 0.5` — เอนจินจะปิด 50% ของโพซิชันเมื่อกำไรถึง +12% บันทึกสถานะ `trade_states.partial_tp_taken` แล้วปล่อยส่วนที่เหลือให้ออกตาม CDC ต่อไป

ประตูความปลอดภัยของ Router: คู่เทรดที่ `mode: live` และเปิด `regime_router_enabled: true` จะถูกบังคับกลับเป็น `sim` โดยอัตโนมัติ เว้นแต่จะมี `regime_router_live_confirmed: true` ด้วย — ห้ามพลิกค่านี้อย่างไม่ผ่าน การอนุมัติจากผู้ดูแลระบบต่อคู่เทรดนั้นๆ โดยตรง ส่วน Regime ที่เป็น NO_TRADE (`PANIC_SELL` , `BEAR_BREAKDOWN` , `BEAR_TREND_STRONG`) จะปิดกั้นการ BUY ใหม่ทั้งหมด, คงการป้องกันของ Stop ไว้, รัด Trailing ให้แคบลงเหลือ 1 เท่าของ ATR และสามารถบังคับปิดโพซิชันได้หลังครบจำนวนแท่งที่กำหนด (`force_close_candles`)

1.4 สิ่งที่ต้องทำ และไม่ทำ

สิ่งที่ต้องทำ

- ประเมินสัญญาณกลยุทธ์บนแท่งเทียนที่ตั้งค่าไว้
- เปิด/ปิดโพซิชันจริงบน Exchange หรือโพซิชันจำลอง
- รักษาการป้องกันด้วย Stop-Loss และเขตความเสี่ยงระดับ Local
- บันทึก State/Event สำหรับ Dashboard, Replay, ตรวจสอบ Incident, และ Telegram

สิ่งที่ต้องไม่ทำ

- ไม่รับประกันกำไร
- ไม่จัดการเงินในกระเป๋า OKX อื่นที่ไม่เกี่ยวข้องกับบัญชี Swap ที่ตั้งค่าไว้
- ไม่ทำให้ปลั๊กอินกลยุทธ์จากภายนอกปลอดภัยอัตโนมัติ เว้นแต่เปิด Sandbox check
- ไม่ทดแทนการตรวจสอบความเสี่ยง, Leverage, ค่าธรรมเนียม, Funding Rate, และกฎของ Exchange ด้วยตัวเองของผู้ใช้

คำแนะนำสำหรับผู้ใหม่:

1. Repository นี้ตั้งค่าไว้สำหรับตลาด Live เดียวคือ OKX `XAUUSD` USDT-settled perpetual swap (CCXT map เป็น `XAU/USDT:USDT`)
2. การรันโหมดจำลอง (`python run_xauby.py --simulate`) ต้องใช้งานได้ก่อนเสมอ ก่อนพิจารณาออเดอร์ Live ใดๆ
3. การเทรด Live ต้องมี "ประตู" 3 อย่างพร้อมกัน: flag `--live` , `LIVE_TRADING=true` , และคู่เทรดที่ Active ตั้ง `mode: live` ใน `coin_whitelist.json`
4. บอกอ่านเฉพาะบัญชี Trading/Swap ของ OKX ไม่ใช่ทุกกระเป๋าใน OKX — ถ้า Dashboard แสดงพอร์ต `0.00 USDT` ให้ตรวจสอบว่าเงินอยู่ในบัญชีที่ใช้เทรด USDT Swap จริง และ API Key มีสิทธิ์อ่าน/เทรดบัญชีนั้น
5. **ห้ามเปิดสิทธิ์ Withdraw บน API Key เด็ดขาด** บอกต้องการแค่สิทธิ์อ่านและเทรดเท่านั้น
6. รันเอนจินเพียง 1 instance ต่อ 1 runtime root — ถ้าต้องการหลายบัญชี ให้ใช้ค่า `XAUBY_HOME` หรือ `XAUBY_INSTANCE_ID` แยกกัน

2.1 การติดตั้ง

สิ่งที่มี: Python 3.12 ขึ้นไป, API Key ของ Exchange ที่ตั้งค่าไว้แล้ว, Telegram Bot (ไม่บังคับ)

```
git clone https://github.com/iisara555/xAuby.git
cd xAuby

python3 -m venv venv
source venv/bin/activate
pip install -r requirements.txt
pip install -e .

cp .env.example .env
python run_xauby.py --simulate
python health_check.py
```

2.2 การตั้งค่า API Key ของ OKX

ก่อนเทรด Live ให้แก้ไข `.env` และใส่ Credential ของ OKX:

```
OKX_API_KEY=...
OKX_API_SECRET=...
OKX_API_PASSPHRASE=...
LIVE_TRADING=true
```

OKX ต้องการทั้ง API Key, Secret, และ Passphrase (ตัวแปรใดตัวแปรหนึ่งจาก `OKX_API_PASSPHRASE`, `OKX_PASSWORD`, หรือ `OKX_API_PASSWORD`) — อย่าคอมมิตไฟล์ `.env` เด็ดขาด

ตรวจสอบคู่เทรดและการตั้งค่า Exchange ก่อนใช้งานจริง:

```
python scripts/evaluate_okx_xau_migration.py --config configs/okx_xau_paper.yaml --whitelist configs/ok
DEFAULT_SYMBOL=XAUUSD python health_check.py
```

2.3 รับโหมดจำลองก่อนเสมอ

โหมดเริ่มต้นของระบบคือ **Simulation** เสมอ ไม่ว่าจะตั้งค่าอะไรผิดพลาด ระบบจะเอนเอียงไปทางปลอดภัย การสลับไป Live ต้องตั้งค่าอย่างชัดเจนตามหัวข้อ 2.5

Entry point อื่นๆ ที่ใช้ได้:

คำสั่ง	ทำอะไร
<code>python run_xauby.py --simulate / --live</code>	เริ่มเอนจินเทรดโดยตรง
<code>python launcher.py</code>	เมนู Launcher แบบโต้ตอบ (เอนจิน + เมนู TUI)
<code>xauby / xauby --sim / xauby --live</code>	Console script ที่ติดตั้งแล้ว (<code>xauby.cli:main</code>)
<code>xauby restart [--live]</code>	รีสตาร์ทแบบควบคุม (สาย Live รับ preflight ผ่านสคริปต์เฉพาะ)
<code>xauby update</code>	ดึงโค้ดจาก GitHub แล้วรีสตาร์ท
<code>python -m xauby.ui.textual_tui.app</code>	เปิด TUI แบบเดี่ยว
<code>python health_check.py</code>	ตรวจสอบสุขภาพระบบครั้งเดียว
<code>bin/xauby</code>	Bash wrapper ที่ cd เข้าโฟลเดอร์โปรเจกต์และเรียก <code>venv/bin/python</code>

2.4 คำสั่งใช้งานที่พบบ่อย

คำสั่งแนะนำสำหรับเริ่ม Live หลังผ่านการจำลองและ Health Check แล้ว:

```
env -u DEFAULT_SYMBOL XAUBY_DEFAULT_SYMBOL=XAUUSDTC XAUBY_FOCUS_SYMBOL=XAUUSDTC \
SIMULATE_ONLY=false BOT_READ_ONLY=false \
./venv/bin/python run_xauby.py --live --pair XAUUSDTC
```

WebUI แบบอ่านอย่างเดียวสำหรับดูจากมือถือ/เบราว์เซอร์:

```
./scripts/start_webui.sh
```

คำสั่งที่ใช้บ่อยระหว่างดูและระบบ:

```
xauby restart          # รีสตาร์ทเอนจิน; โหมด Live รับ preflight แบบควบคุม
xauby restart --live   # บังคับ restart แบบควบคุมสำหรับ Live
xauby update           # ดึง origin/main จาก GitHub แล้วรีสตาร์ท
```

`xauby update` เป็นทางเลือกของคำสั่ง:

```
scripts/deploy_from_github.sh --restart --branch=main
```

เส้นทาง Controlled Restart จะยอมรับโพซิชั่นที่บอกติดตามอยู่แล้ว (รวมออเดอร์ Stop-Loss ที่ตรงกัน) แต่จะปิดกันเมื่อพบยอดคงเหลือที่ไม่รู้จักหรือออเดอร์เปิดที่ไม่ถูกติดตาม

2.5 เช็กलिस्टก่อนเปิดใช้งานจริง (Live)

การเทรด Live ต้องผ่านทุกเงื่อนไขพร้อมกัน:

1. flag `--live` ตอนรันคำสั่ง

2. ตัวแปรแวดล้อม `LIVE_TRADING=true`
3. `simulate_only: false` ใน YAML
4. คู่เทรดนั้นตั้ง `mode: live` ใน whitelist

โหมด	วิธีเปิด	พฤติกรรม
Simulation	<code>--simulate</code> , <code>simulate_only: true</code> , หรือ per-asset <code>mode: sim</code>	ไม่มีออเดอร์จริง; ใช้บัญชี SimBroker เมื่อเปิดใช้งาน
Live	<code>--live</code> + <code>LIVE_TRADING=true</code> + <code>simulate_only: false</code> + per-asset <code>mode: live</code>	ออเดอร์จริงบน Exchange
Read-only	<code>--read-only</code> หรือ <code>read_only: true</code>	ประมวลสัญญาณอย่างเดียว ไม่ส่งออเดอร์
Semi-auto	<code>trading.mode: semi_auto</code>	รอยืนยันผ่าน Telegram ก่อน BUY
Telegram Pause	<code>/pause</code> แล้วกด Confirm PAUSE	บล็อกการ BUY ใหม่ทั้งระบบ; <code>/resume</code> ปลดล็อก

รายการเช็กलिस्टก่อน Live แบบละเอียดทั้งหมด (รวมถึงเรื่อง Risk, Replay Validation, Regime Router Live-confirm) อยู่ที่หัวข้อ **11.4** ทำเล็ม — ควรอ่านให้ครบก่อนเปิดเงินจริง

3.1 ภาพรวมโครงสร้างเลเยอร์

xAuby แบ่งเป็น 5 เลเยอร์ที่ทำงานร่วมกัน:

เลเยอร์	ส่วนประกอบหลัก	บทบาท
Operator	<code>launcher.py</code> , <code>run_xauby.py</code> , Textual TUI, Telegram	จุดเริ่มต้นและควบคุมจากมนุษย์
Core Runtime	<code>LiteTradingEngine</code> , <code>PairRegistry</code> , <code>SymbolContext</code> , <code>RegimeRouter</code> , <code>StrategyRunner</code> , Risk/PreTradeGate, Order Executor/SimBroker	แกนกลางของระบบเทรด
Strategy Plane	Strategy plugins, Indicator plugins, Strategy/Portfolio Resolver	ตรรกะสัญญาณและการแสดงผลกราฟ
Data Plane	CCXT REST client, WebSocket, LiteDB/SQLite	ดึงข้อมูลตลาดและเก็บ state
Observability	EventEmitter, JSONL events, State JSON, HealthMonitor	บันทึกเหตุการณ์เพื่อ Replay/ตรวจสอบ/แสดงผล

ลำดับการไหลของข้อมูล: Launcher/CLI เรียก Engine → Engine ดึง Pair จาก Registry → สร้าง SymbolContext ต่อคู่เทรด → ผ่าน RegimeRouter (ถ้าเปิดใช้) → StrategyRunner ประเมินสัญญาณ → ผ่าน Risk/PreTradeGate → ส่งออเดอร์จริงหรือจำลอง → บันทึกลง DB และ Event → อัปเดต State JSON ให้ TUI/WebUI อ่าน

3.2 หนึ่ง Tick ของเอนจินทำอะไรบ้าง

เอกสารนี้อธิบายการทำงานหนึ่ง "Tick" ของ `LiteTradingEngine` สำหรับหนึ่งคู่เทรด โดย Tick ขึ้นนอกจะวนลูปทุกคู่เทรดที่ Active จาก PairRegistry

ลำดับขั้นตอนต่อคู่เทรด

1. Sync แท่งเทียนผ่าน REST
2. ตรวจสอบว่าราคาจาก WebSocket เก่าเกินไปหรือไม่ — ถ้าใช่ ใช้ REST ticker แทน, ถ้าไม่ ใช้ค่าจาก WS
3. อ่านสถานะการเทรดปัจจุบัน (Trade State)
4. Resolve การตั้งค่างลยุทธ์และพอร์ตของคู่เทรดนั้น
5. ถ้า RegimeRouter Active สำหรับคู่นี้: จำแนก Regime + Debounce + เลือกเส้นทาง; ถ้าไม่: ใช้กลยุทธ์ที่ตั้งค่าไว้ตรงๆ
6. ถ้า Regime เป็น NO_TRADE: บล็อกการ BUY ใหม่และรัด Protection ให้แน่นขึ้น; ถ้าไม่: ให้ StrategyRunner ประเมินสัญญาณ
7. จัดการโพซิชันเดิม / recovery / บังคับปิด (ถ้าอยู่ในเงื่อนไข NO_TRADE)
8. ตรวจสอบ Macro Guard ก่อนอนุญาต BUY — ถ้าบล็อก จะ HOLD พร้อมแจ้งเตือน

9. ตามสถานะ+Action: `idle+BUY` → เปิดไม้ / คิว semi-auto, `bought+SELL` → ปิดไม้, `bought+HOLD` → ประมวล Partial TP / Trailing SL / Breakeven
10. บันทึก Trade State + Event ทั้งหมด
11. อัปเดต State JSON ให้ TUI อ่านต่อ

เส้นทางเข้าไม้ (Entry Path)

1. Protection ตรวจสอบไม้ต่อวัน, % ขาดทุนต่อวัน, จำนวนแพ้ติดต่อกัน, และจำนวนโพสิชันเปิดสูงสุด
2. PreTradeGate ตรวจสอบ Min Notional และ Slippage เทียบราคาสัญญา
3. คำนวณขนาดไม้จาก Config พอร์ตของผู้เทรดนั้น พร้อมเพดาน Deploy ระดับ Global
4. เส้นทางจอเดอร์ใช้ Live Executor หรือ SimBroker ตามโหมดของผู้เทรดนั้น
5. วาง Stop-Loss บน Exchange (โหมด Live) หรือติดตาม/จำลองใน Local (โหมด Sim)
6. ส่งรายงานผ่าน Telegram พร้อม Tag สัญลักษณ์และโหมด

เส้นทางออกไม้ (Exit Path)

ทริกเกอร์การออกจากโพสิชันมีได้จาก: สัญญา SELL จากกลยุทธ์, Partial Take-Profit แบบ one-shot, Stop-Loss ที่ Local ยืนยันครบจำนวน tick, การตรวจพบ SL Fill บน Exchange, RegimeRouter บังคับปิดเมื่อ NO_TRADE ครบจำนวนแท่ง, หรือ `max_hold_hours` / ตาราง Minimal ROI (ถ้าตั้งค่าไว้) หากการขายล้มเหลว ระบบจะพยายามกู้คืน Stop-Loss และส่ง Telegram แจ้งเตือนระดับวิกฤตถ้ากู้คืนไม่สำเร็จ

เส้นทาง Partial Take-Profit

Partial TP จะทำงาน**หลัง**ที่ pipeline การออกไม้เต็มมีโอกาสขายก่อนเสมอ — ถ้า Tick เดียวกัน เกิด Full Exit ด้วย Full Exit จะชนะ มิฉะนั้น สำหรับสถานะ `bought` ที่ยังไม่เคยเก็บ Partial TP เอนจินจะเทียบราคา Mark กับเกณฑ์ที่ตั้งจากราคาเข้า:

- LONG: $\text{mark} \geq \text{entry} \times (1 + \text{partial_tp_pct} / 100)$
- SHORT: $\text{mark} \leq \text{entry} \times (1 - \text{partial_tp_pct} / 100)$

เมื่อถึงเกณฑ์ `execute_partial_tp()` จะปิดสัดส่วนที่ตั้งไว้ผ่าน Broker Abstraction, ปรับขนาด Stop-Loss ฝั่ง Exchange (ถ้ามี) ให้ตรงกับจำนวนที่เหลือ, ตั้ง `partial_tp_taken=true`, ส่ง Event `partial_tp_executed` แล้วปล่อยส่วนที่เหลือให้ออกตามกลยุทธ์/CDC ต่อไปตามปกติ

การส่งมอบกลยุทธ์ (Strategy Handoff)

เมื่อ RegimeRouter เปลี่ยนกลยุทธ์ขณะมีโพสิชันเปิดอยู่ ระบบออกแบบให้ปลอดภัยเสมอ:

- โพสิชันเดิมยังคงเป็นของกลยุทธ์เก่าจนกว่าจะปิด
- ไม้ใหม่จะถูกบล็อกหรือรอจนกว่า Handoff จะเสร็จสมบูรณ์
- Handoff จะเสร็จสมบูรณ์ก็ต่อเมื่อโพสิชันปิดแล้วเท่านั้น
- เอนจินส่ง Event สำหรับ Handoff เพื่อการตรวจสอบย้อนหลัง

สิ่งนี้ป้องกันไม่ให้กลยุทธ์ใหม่มาบริหารการออกของโพสิชันที่ตัวเองไม่ได้เป็นคนเปิด

การจัดการ NO_TRADE

สำหรับ Regime ที่แม็ฟไปเป็น `None` : บล็อกการ BUY ใหม่ทั้งหมด, คง Stop Protection เดิมไว้, รัด Trailing Stop ให้แคลง เหลือ 1 เท่า ATR (ถ้ามี), ต้องผ่านแท่งที่เทรดได้ครบตาม `regime_router.recovery_candles` ก่อนจะกลับมาเทรดได้ปกติ, และสามารถบังคับปิดได้หลังครบ `regime_router.force_close_candles` แท่งที่เป็น NO_TRADE ติดต่อกัน ปัจจุบัน Regime ที่เป็น NO_TRADE คือ `PANIC_SELL`, `BEAR_BREAKDOWN`, และ `BEAR_TREND_STRONG`

การเทรด Short

กลยุทธ์ที่รองรับ Short จะส่งสัญญาณ `OPEN SHORT` / `CLOSE SHORT` แทนที่จะยึดเข้าไปใน BUY/SELL บน USDT Perpetual การเปิด Short จะแม็ฟเป็น SELL และการปิดแม็ฟเป็น BUY แบบ Reduce-only Stop-Loss จะอยู่เหนือราคาเข้า และ Fixed Take-Profit จะอยู่ต่ำกว่าราคาเข้า การกระหนยอดโพชิชัน (Reconciliation) ใช้ Endpoint โพชิชันของ Exchange โดยตรง (ไม่ใช่ ยอดคงเหลือแบบ Spot) และจะบล็อกการเทรดถ้าพบ โพชิชันกำพำหรือฝ่งไม่ตรงกัน

3.3 PairRegistry และ SymbolContext

`PairRegistry` รวมข้อมูลจาก 3 แหล่ง: (1) สิ้นกรัฟฟิใน `coin_whitelist.json` (กลยุทธ์, โหมด, Timeframe ต่อเหรียญ), (2) `data.pairs` ใน YAML เมื่อจำเป็นต้องใช้รายชื่อแบบเก่า/ชัดเจน, และ (3) `DEFAULT_SYMBOL` จาก environment (ถ้ามี) เมื่อ `architecture.whitelist_strict: true` Whitelist จะเป็นแหล่งความจริงเดียว และเมื่อ `hot_reload_enabled: true` ระบบจะ poll mtime ของไฟล์ทุก `reload_interval_seconds` วินาที เมื่อไฟล์เปลี่ยน เอนจิ้นจะ re-init `SymbolContext` และรีเฟรช WebSocket subscription ใหม่

แต่ละคู่เทรดที่ Active จะได้ `SymbolContext` ของตัวเอง ประกอบด้วย:

- Snapshot ราคาจาก WebSocket พร้อม REST fallback
- กลยุทธ์ปัจจุบันและเป้าหมาย Handoff (ถ้ามี)
- Regime ปัจจุบันและ Snapshot ล่าสุด
- โหมดการทำงานต่อคู่เทรด (`sim` หรือ `live`)
- สถานะรอยืนยันของ Semi-auto
- Metadata สัญญาณล่าสุดสำหรับ TUI และ Telegram

คู่เทรดถูกแยกจากกันในระดับ Strategy/Runner/Context แต่เงินทุนพอร์ตและ Guard ระดับ Global ยังใช้ร่วมกันโดยเจตนา

3.4 กฎการพึ่งพาของระบบ (Dependency Rules)

กฎ	เหตุผล
<code>observability</code> ห้าม import <code>engine</code>	Replay และ Health ต้องไม่ผูกกับเอนจิน
TUI อ่านอย่างเดียว (<code>LiteDB(readonly=True)</code>)	หน้าจอตองไม่บล็อกหรือแก้ไขข้อมูลการเทรด
Secret อยู่ใน <code>.env</code> เท่านั้น	<code>bot_config.yaml</code> คอมมิตได้ปลอดภัยเพราะไม่มีคีย์ใดๆ
เอนจิน 1 instance เท่านั้น	<code>core/.engine.lock</code> ป้องกันการเทรดซ้ำซ้อน
Strategy plugin กับ Indicator plugin ต้องไปด้วยกันเสมอ	กราฟ/Legend ต้องตรงกับพฤติกรรมกลยุทธ์จริง
เอนจินต้อง Strategy-agnostic	ตรรกะสัญญาณ/ทางออกอยู่ที่ Config/Plugin เท่านั้น

3.5 ปลั๊กอิน Strategy และ Indicator

นี่คือรูปแบบขยายระบบที่สำคัญที่สุด: **Strategy และ Indicator ต้องมาคู่กันเสมอ** PR ที่เพิ่ม Strategy โดยไม่มี Indicator plugin คู่กันและไม่มี mapping ใน `strategy_chart_indicators` ถือว่ายังไม่สมบูรณ์ เพราะ Dashboard จะแสดงผลไม่ตรงกับพฤติกรรมจริง

Strategy Plugin

อยู่ที่ `xauby/strategies/<name>/strategy.py` — สืบทอดจาก `xauby.strategies.base.Strategy`, ประดับด้วย `@register("<name>")`, Implement `analyze(ctx: MarketContext) -> Signal` (และ `default_config()` / `validate_config()` แบบไม่บังคับ) Strategy ต้องเป็น "นักวิเคราะห์หีบห่อ" เท่านั้น — ห้ามวางออเดอร์, แตะ DB, เรียก Telegram, หรือเรียกเมธอดของเอนจินโดยตรง เอนจินจะโหลดปลั๊กอินอัตโนมัติผ่าน `load_strategy(name, config)`

Indicator Plugin

อยู่ที่ `xauby/strategies/indicators/indicator_<name>.py` — สืบทอดจาก `Indicator` base class, `compute(df, config)` เติมคอลัมน์ที่มีชื่อ (ห้ามมี ANSI code), `display_config` กำหนดโซน/เส้น/เมตริก/ป้ายชื่อ/สี ป้อนเข้ากราฟเทอร์มินัล และ Legend/Checklist ของ TUI

เช็คลิสต์การเพิ่ม Strategy ใหม่

1. Strategy plugin + `@register`
2. Indicator plugin + `@register` (หรือใช้ตัวที่เข้ากันได้อยู่แล้ว)
3. เพิ่ม `bot_config.yaml -> architecture.strategy_chart_indicators.<strategy>`
4. เขียนเทสต์: strategy test, indicator test, และ chart legend coverage

กลยุทธ์กลุ่ม `*_short` (`donchian_short`, `supertrend_short`, `rsi2_short`) และปลั๊กอินวิจัยอื่นๆ (`rsi2_meanrev`, `vol_breakout`) ถูก tag ว่า `research` — เอนจินจะ**บล็อกอัตโนมัติ**ไม่ให้กลยุทธ์ที่ tag `research` รับบนคู่เทรดที่ Live เกิดขาด

(`_load_strategy_for_symbol`) จึงใช้ได้แค่ในโหมด Sim/Backtest เท่านั้น ห้าม map เข้า `regime_router.mapping` เพื่อ production

3.6 Feature Flags ทั้งหมด (architecture.*)

ฟีเจอร์ระดับสถาปัตยกรรมทุกตัวถูกควบคุมด้วย Flag ใต้ `bot_config.yaml -> architecture:` การ Rollback คือการพลิก Flag กลับเป็น `false` — ไม่ต้อง revert โค้ด

Flag	หน้าที่
<code>whitelist_strict</code>	Whitelist เป็นแหล่งความจริงเดียวของคู่เทรด/กลยุทธ์ พร้อม fail-fast validation
<code>indicator_registry_enabled</code>	กราฟคำนวณ Indicator ผ่าน Registry แทนการ hard-code
<code>sync_yaml_pairs_from_whitelist</code>	ซิงค์ <code>data.pairs</code> จาก whitelist อัตโนมัติ
<code>tui_indicator_registry</code>	Checklist/Legend ของ TUI มาจาก Registry Adapter
<code>per_symbol_execution_mode</code>	แต่ละคู่เทรดเลือก <code>sim / live</code> แยกกันได้
<code>sim_broker_enabled</code>	ออเดอร์จำลองผ่าน SimBroker แบบมี Ledger จริง
<code>regime_router_enabled</code>	สวิตช์หลักของ RegimeRouter (ยังต้องเปิดต่อคู่เทรดด้วย)
<code>regime_confidence_filter</code>	ระงับการสลับ Regime เมื่อ Confidence ต่ำกว่าเกณฑ์
<code>regime_statistical_crosscheck</code>	เปิดตัวจับ Regime เชิงสถิติแบบ GMM คู่ขนาน (advisory เท่านั้น ดูหัวข้อ 5.4)
<code>event_bus_enabled</code>	ส่ง Event ของ EventEmitter ให้ Subscriber ภายในโปรเซส
<code>exchange_plugin_registry_enabled</code>	Deprecated/ไม่มีผลจริงแล้ว — Gateway ใช้ Registry เสมอ; เก็บไว้เพื่อ backward-compat
<code>strategy_sandbox_strict</code>	ปฏิเสธ (ไม่ใช่แค่เตือน) กลยุทธ์ที่ไม่ผ่าน Static Sandbox Scan
<code>strategy_validate_strict</code>	ปฏิเสธกลยุทธ์ที่ <code>config_errors()</code> ไม่ว่างตอนโหลด

4.1 ไฟล์ตั้งค่า 3 ไฟล์หลัก

ไฟล์	คอมมิตไหม	หน้าที่
<code>.env</code>	ไม่	API key, <code>LIVE_TRADING</code> , Telegram
<code>.env.example</code>	ใช่	เทมเพลตสำหรับติดตั้งใหม่
<code>bot_config.yaml</code>	ใช่	Engine, Strategy, Portfolio, การแจ้งเตือน, retention, optimizer
<code>coin_whitelist.json</code>	ใช่	คู่เทรด Active, Timeframe, กลยุทธ์ต่อคู่, โหมด, ประตูลูก Router

ไฟล์ Runtime ใต้ `core/` (DB, log, lock, ยอด sim, cache backtest) เป็นไฟล์เฉพาะเครื่อง ห้ามคอมมิตและห้ามถือว่าเป็นค่าตั้งค่าที่ Portable ข้ามเครื่อง เว้นแต่ตั้งใจจะคัดลอกทั้ง runtime root

4.2 หลักการ Source of Truth 3 ชั้น

Config ถูกแบ่งความรับผิดชอบเป็น 3 ชั้น การใส่ค่าผิดชั้นคือสาเหตุที่มักเกิดข้อผิดพลาดที่สุด เพราะ Backtest, Live, Replay, และ Optimizer ทั้งหมด resolve ผ่าน resolver ตัวเดียวกัน — ถ้าค่าไม่ตรงชั้น จะทำให้ผลไม่ตรงกัน (parity พัง)

ชั้น	เป็นเจ้าของ	ห้ามเป็นเจ้าของ
Engine Config (<code>bot_config.yaml</code> ระดับบนสุด)	Exchange, Logging, Monitoring, การแจ้งเตือน, Global risk guard, Feature flag	พารามิเตอร์ สัญญาณ/ทางออก ของกลยุทธ์
Strategy Config (<code>strategy.config.<id></code> , <code>strategy.symbols.<SYMBOL></code> , whitelist <code>strategy_params</code>)	RSI/ATR/Volume, SL, Trailing, Breakeven, Timeframe กลยุทธ์, ตรรกะ สัญญาณ/ทางออก	Credential หรือ ขนาดไม้
Portfolio Config (<code>portfolio.position_sizing</code> , <code>portfolio.symbols.<SYMBOL>.position_sizing</code>)	ขนาดไม้, การจัดสรร, เงินทุน	กฎ Indicator/ สัญญาณ

กฎทองคำ: ค่าใดก็ตามที่เปลี่ยนผลลัพธ์ Backtest ได้ ต้องอยู่ใต้ Strategy หรือ Portfolio Config เท่านั้น ห้ามอยู่ใน Engine-only block เอนจินต้อง Strategy-agnostic เสมอ

4.3 การตั้งค่า Exchange (OKX ผ่าน CCXT)

`bot_config.yaml -> exchange:` คือแหล่งความจริงเดียวของการเลือก Exchange และตัวเลขเศรษฐศาสตร์ระดับ Exchange เอนจิน, Backtest, Replay, Health Probe, และสคริปต์ปฏิบัติการทั้งหมด resolve ผ่าน Helper ชุดเดียวกัน (`xauby/runtime/exchange_config.py`) ทำให้ Simulation คงความสอดคล้องกับ Live เสมอ

```

exchange:
  provider: ccxt
  ccxt_id: okx
  name: okx
  quote_asset: USDT
  settle_asset: USDT
  fee_pct: 0.0005
  market_type: swap
  margin_mode: isolated
  position_mode: one_way
  api_key_env: OKX_API_KEY
  api_secret_env: OKX_API_SECRET
  base_url_env: OKX_BASE_URL
  params:
    options:
      defaultType: swap

```

- **Credential** อ่านจากตัวแปรแวดล้อมที่ผูกกับ Exchange OKX ต้องมี `OKX_API_KEY` , `OKX_API_SECRET` , และหนึ่งใน `OKX_API_PASSPHRASE` / `OKX_PASSWORD` / `OKX_API_PASSWORD`
- **ลำดับความสำคัญของค่าธรรมเนียม (Fee):** `exchange.fee_pct` → `backtest.fee_pct` → `trading.fee_pct` → `0.001` ค่า `sim_fee_pct` ต่อคู่เทรด (ใน whitelist) จะชนะสำหรับคู่นั้นโดยเฉพาะ
- CCXT คือ Adapter หลักที่ใช้จริงสำหรับ OKX

4.4 โครงสร้าง Whitelist และคู่เทรดปัจจุบัน

```

{
  "version": 1,
  "quote_asset": "USDT",
  "assets": [
    {
      "symbol": "XAU",
      "enabled": true,
      "primary_timeframe": "4h",
      "confirm_timeframe": "1d",
      "strategy": "cdc_action_zone",
      "mode": "live",
      "allowed_sides": ["long", "short"],
      "leverage": 1.0,
      "short_live_enabled": true,
      "regime_router_enabled": false,
      "regime_router_live_confirmed": false,
      "sim_fee_pct": 0.05
    }
  ]
}

```

ฟิลด์	ค่าที่เป็นไปได้	หมายเหตุ
<code>symbol</code>	สินทรัพย์ฐาน เช่น <code>XAU</code>	ระบบจะต่อท้ายด้วย <code>quote_asset</code> อัตโนมัติ
<code>enabled</code>	bool	สินทรัพย์ที่ปิดจะถูกข้ามไป
<code>primary_timeframe</code>	เช่น <code>1h</code> , <code>4h</code>	ใช้ทั้ง Live, Backtest, และกราฟ
<code>confirm_timeframe</code>	เช่น <code>1d</code> หรือว่าง	ขึ้นกับกลยุทธ์
<code>strategy</code>	plugin id	จำเป็นเมื่อ <code>whitelist_strict: true</code>
<code>strategy_params</code>	object	ค่าปรับแต่งเฉพาะคู่เทรด รวมเข้ากับ Strategy Config
<code>mode</code>	<code>sim</code> , <code>live</code>	ต้องเปิด <code>per_symbol_execution_mode: true</code>
<code>allowed_sides</code>	array เช่น <code>["long"]</code> , <code>["long", "short"]</code>	ฝั่งที่อนุญาตให้เปิดไม้ต่อคู่เทรด — ต้องมี <code>"short"</code> ก่อนกลยุทธ์ Short จะทำงานได้ (ดูหัวข้อ 6.5)
<code>leverage</code>	number	เลเวอเรจของคู่เทรดนั้น ต้องไม่เกิน <code>derivatives.max_leverage</code> ระดับ Engine
<code>short_live_enabled</code>	bool	ประตูเพิ่มเติมสำหรับ Short บนเงินจริง — ถ้าปิด Short จะเป็น Paper-only แม้ <code>allowed_sides</code> จะมี <code>"short"</code> ก็ตาม
<code>regime_router_enabled</code>	bool	ประตู RegimeRouter ระดับคู่เทรด
<code>regime_router_live_confirmed</code>	bool	ต้องเป็น true ก่อนคู่เทรด Live จะ Route กลยุทธ์ได้
<code>sim_fee_pct</code>	number	หน่วยเปอร์เซ็นต์ เช่น <code>0.05</code> = 0.05%

หมายเหตุ: `strategy_params` ไม่ได้แสดงในตัวอย่าง JSON ข้างบนเพราะเป็น Field เสริม (Optional) — ดูตัวอย่างค่าจริงที่คู่ XAU ใช้ในหัวข้อ 5.2

4.5 ตัวแปรแวดล้อม (Environment Variables)

ตัวแปร	ค่าเริ่มต้น	คำอธิบาย
OKX_API_KEY / OKX_API_SECRET / OKX_API_PASSPHRASE	-	Credential สำหรับ OKX REST/WS
LIVE_TRADING	false	ประตูปิดสำหรับออเดอร์จริง
SIMULATE_ONLY	จาก YAML	CLI --live / --simulate override
BOT_READ_ONLY	false	ห้ามการวางออเดอร์
TELEGRAM_ENABLED	false	สวิตช์หลักของการแจ้งเตือน
TELEGRAM_BOT_TOKEN	-	Token จาก @BotFather
TELEGRAM_CHAT_ID	-	Chat ที่ได้รับอนุญาตให้ส่งงาน
DEFAULT_SYMBOL	-	Override คู่เทรดเดี่ยว (ถ้ามี)
XAUBY_HOME	core	โฟลเดอร์ฐานของ Runtime data — ย้ายเพื่อรันหลาย instance คู่กันได้
XAUBY_INSTANCE_ID	-	Sub-namespace ใต้ Runtime root (เช่น tenant/account id)
XAUBY_DEFAULT_SYMBOL	XAUTUSD	สัญลักษณ์ Dashboard ส่องสุดท้ายเมื่อ Whitelist ว่าง/อ่านไม่ได้ (Asset-neutral — ไม่ผูกกับทองคำตายตัว)
XAUBY_FOCUS_SYMBOL	-	บังคับคู่เทรดที่ TUI/Dashboard โฟกัสตอนเริ่มทำงาน ใช้คู่กับ DEFAULT_SYMBOL ตอน Live Startup (ดูหัวข้อ 1.3/11.1)
OKX_BASE_URL	auto	Override Endpoint REST ของ OKX (ชื่อตัวแปรมาจาก exchange.base_url_env)
BINANCE_BACKTEST_URL	api.binance.com	Override แหล่งข้อมูล Backtest จาก Global Binance (ดูหัวข้อ 7.1)

คำสั่ง "Recommended live startup" ในหัวข้อ 1.3/11.1 สั่ง `env -u DEFAULT_SYMBOL` (unset ตัวแปร Generic เดิม) แล้วตั้ง `XAUBY_DEFAULT_SYMBOL` และ `XAUBY_FOCUS_SYMBOL` แทน เพื่อกันไม่ให้ `DEFAULT_SYMBOL` ที่อาจตั้งค้างไว้จาก Session อื่นมาแย่ง Override สัญลักษณ์ที่ตั้งใจใช้จริง

น้ำหนัก Macro ของ Regime Classifier

ตัวจำแนก Regime รวม DXY, อัตราดอกเบี้ย, และข่าวเป็น Macro Bias น้ำหนักปรับได้เพื่อไม่ให้ระบบผูกติดกับทองคำ เพียงอย่างเดียว:

```
macro_sentiment_guard:
  macro_weights: # ค่าเริ่มต้น (สมมติฐานสำหรับทองคำ)
    news: 1.0
    dxy: 1.0      # ค่าถูก orient ไว้แล้วที่ producer (บวก=bullish) ห้าม invert ซ้ำ
    fred: 1.0
  regime_bias_threshold: 0.3 # |combined_macro| เกินค่านี้จึงพลิกเป็น RISK-ON/RISK-OFF
```

4.6 เมนู Quick Config ในตัว

Launcher menu → **Quick Configuration** (หรือ `xauby --config`) เปิด Editor แบบ Textual: เมนู `OptionList` แบบ จัดกลุ่มที่คลิกเข้าสู่ Submenu ของ Toggle, Modal ตัวเลข/ช่วงค่า, และตัวเลือกแบบเลือก (นำทางด้วยลูกศร/Enter/เมาส์ ค่า secret จะถูกซ่อน) การเขียนค่ายังผ่าน `ruamel.yaml` round-trip เสมอ ทำให้ **คอมเมนต์และโครงสร้างใน `bot_config.yaml` ไม่หายไประหว่างแก้ไข**

กลุ่ม	แก้ไขอะไรได้
Trading & Risk	โหมด (sim/live), % ความเสี่ยงต่อคู่, SL ATR, Breakeven, ตัวกรอง D1, กลยุทธ์
Global Trading	<code>max_open_positions</code> , <code>interval_seconds</code> , timeframe, min order amount, max position per trade
Risk Guards	Drawdown guard, max daily loss %, max consecutive losses
Portfolio	Initial balance, allocation % ต่อคู่เทรด
Strategy Params	RSI min/max, vol min ratio, trailing ATR mult, cool down ต่อคู่เทรด
Regime Router	เปิด/ปิด, confidence threshold, debounce/recovery/force-close, mapping (เลือกจากกลยุทธ์ที่ลงทะเบียนแล้ว)
Multi-Pair & Dashboard	Timeframe, รายชื่อคู่เทรด, focus ของ Dashboard
Telegram Alerts & Schedule	Credential/test, กำหนดการ weekly/daily digest
Macro Guard	Sentiment guard, FRED, AI news
Exchange	Wizard สลับ Exchange แบบมีโค้ด, credential ต่อ Exchange, ทดสอบการเชื่อมต่อ
System Features	Architecture flags ของ per-symbol execution และ event bus

5.1 กลยุทธ์ (Strategy Plugin) ทั้งหมดในระบบ

ระบบมีปลั๊กอินกลยุทธ์ทั้งหมด 14 ตัว บางตัวใช้งานจริง (Production) บางตัวเป็นงานวิจัย (Research/Sim เท่านั้น):

Strategy	Indicator plugin	โชนบบกราฟ	เส้นบบกราฟ
<code>cdc_action_zone</code> ★Live	<code>cdc_action_zone</code>	น้ำเงิน/เขียว/แดง/กลาง	EMA12, EMA26
<code>supertrend_ema200</code>	<code>supertrend</code> , <code>ema200</code>	ST Bull/Bear/กลาง	SuperTrend, EMA200
<code>bbkc_squeeze</code>	<code>bbkc_squeeze</code>	บีบตัว/ทะลุ/กลาง	Bollinger Bands, Keltner Channels
<code>bbrsi_mean_reversion</code>	<code>bbrsi_mean_reversion</code>	Oversold/ กลาง/Overbought	Bollinger Bands
<code>btc_ema_pullback</code>	<code>btc_ema_pullback</code>	เทรนด์/ पुलเบ็ก/ ยึดกลับ/ กลาง	EMA Fast/Slow/Trend
<code>ict_lite_strategy</code>	<code>ict_lite</code>	Sweep Low/Reclaim/MSS/ กลาง	EMA Fast/Slow, High/Low ล่าสุด
<code>rsi2_meanrev</code> (research)	<code>rsi2_meanrev</code>	Buy Setup/Oversold/Exit/ กลาง	EMA200, SMA5
<code>sol_ema_pullback</code>	<code>sol_ema_pullback</code>	เทรนด์/ पुलเบ็ก/ ยึดกลับ/ กลาง	EMA20, EMA50
<code>vol_breakout</code> (research)	<code>vol_breakout</code>	ทะลุ/ATR ขยายตัว/กลาง	Range High, Exit EMA
<code>xauby_vwap_pullback</code>	<code>xauby_vwap_pullback</code>	เข้า/ पुलเบ็ก/เทรนด์/ร่อ/ เสียเทรนด์	EMA20/50/200, VWAP
<code>donchian_trend</code> (candidate)	-	-	Turtle breakout เหนือ EMA200 + ADX filter — ผู้ทำซิง ที่ยังไม่ผ่านเกณฑ์คัดเลือก ไม่ได้อยู่ ใน production mapping (ดู 5.7)
<code>donchian_short</code> / <code>supertrend_short</code> / <code>rsi2_short</code> (research)	-	Short-side เฉพาะงานวิจัย ไม่ใช่ Production	

★ = กลยุทธ์ที่รันอยู่บนคู่เทรด Live ปัจจุบัน (XAUUSD)

5.2 CDC Action Zone — กลยุทธ์หลักที่ใช้งานจริง

`cdc_action_zone` เป็นกลยุทธ์เดียวที่รันบนคู่เทรด Live ปัจจุบัน ทำงานแบบ **Stop-and-reverse** คือเปิด Long หรือ Short สลับกันตามการกลับสียงโซน (Zone Flip) ไม่มี Stop-Loss ฝั่ง Exchange (`disable_stop_loss: true`) — การออกจากไม้

ทั้งหมดขับเคลื่อน ด้วยการเปลี่ยนโซนเท่านั้น ขนาดไม้คำนวณจาก `position_pct` คงที่ของ Equity (ไม่ใช่ระยะ SL) และมี Partial Take-Profit แบบ one-shot ที่ +12% เก็บ 50% ของโพสิชัน

พารามิเตอร์ (จาก whitelist จริง)	ค่า
<code>sl_atr_mult</code>	2.0
<code>trailing_atr_mult</code>	1.8
<code>breakeven_sl_enabled</code>	false
<code>require_fresh_zone</code>	true (fresh_zone_window: 1)
<code>require_slow_slope</code>	true (slow_slope_bars: 3)
<code>position_pct</code>	0.95
<code>partial_tp_pct</code> / <code>partial_tp_fraction</code>	12.0 / 0.5
<code>backtest_data_proxy</code>	PAXGUSD (ใช้แทนทองในการ Backtest)

5.3 Market Regime Detector คืออะไร ทำงานอย่างไร

Market Regime Detector (`xauby/regime/classifier.py` , ฟังก์ชัน `classify_market()`) คือตัวจำแนกสภาพตลาดแบบ Rule-based ที่วิเคราะห์ตลาดผ่าน 4 แกนหลัก แล้วสังเคราะห์เป็น "Regime" 11 ประเภท ตามแนวคิด Wyckoff (สะสม → ขาขึ้น → กระจาย → ขาลง):

4 แกนที่ใช้วิเคราะห์

- **Trend** — จาก EMA12/EMA26/EMA200 stack + ระยะห่างราคาจาก EMA200 + Momentum 5/20 แก่ง
- **Volatility** — ATR เทียบราคา แล้วจัดอันดับเทียบค่าเฉลี่ยย้อนหลัง **120 แก่ง** (Relative ไม่ใช่ Absolute — จุดสำคัญที่ทำให้ใช้ข้ามสินทรัพย์/ยุคสมัยได้)
- **Liquidity** — Volume Ratio เทียบ SMA 20 แก่ง
- **Macro** — รวม DXY + อัตราดอกเบี้ย (FRED) + Sentiment ข่าว โดย Clamp แต่ละค่าให้อยู่ในช่วง [-1, 1] ก่อนรวม (ป้องกันค่าใดค่าหนึ่ง เช่น News Score จาก AI ที่ไม่ถูก Clamp ตีทาง มา Dominate ผลลัพธ์)

รายชื่อ Regime ทั้ง 11 ประเภท

Regime	ความหมาย	Phase
BULL_BREAKOUT	ขาขึ้นทะลุพร้อม Volume ยืนยันและ Volatility ขยายตัว	MARKUP
BULL_TREND_STRONG	ขาขึ้นแข็งแกร่ง Momentum/EMA spread สูง	MARKUP
BULL_TREND_WEAK	ขาขึ้นอ่อนแรงหรือ Macro ขัดแย้ง	MARKUP/RECOVERY_RALLY
BEAR_BREAKDOWN	ขาลงทะลุพร้อม Volatility ขยายตัว	MARKDOWN
BEAR_TREND_STRONG	ขาลงแข็งแกร่ง	MARKDOWN
BEAR_TREND_WEAK	ขาลงอ่อนแรง	DISTRIBUTION
PANIC_SELL	เทขายตื่นตระหนก — ขาลง+Volatility พุ่ง+Macro หรือ Momentum ลมรุนแรง	CAPITULATION
LOW_VOL_ACCUMULATION	Volatility ต่ำ ราคาแนบ EMA200 — ช่วงสะสม	ACCUMULATION
LOW_VOL_RANGE	Volatility ต่ำ แต่ไม่แนบ EMA200 พอ	ACCUMULATION
VOLATILITY_EXPANSION	Volatility ขยายตัวโดยไม่มีเทรนด์ชัดเจน + Range กว้าง	TRANSITION
SIDEWAYS_CHOP	ไม่เข้าเงื่อนไขไหนเลย — แกว่งไร้ทิศทาง	RANGE

Confidence Score

ทุก Regime มาพร้อม Confidence (0.05–0.98) ที่คำนวณแบบ Composite จากหลายปัจจัย: Trend Quality (34%), Macro Alignment (20%), Macro Conviction (14%), Vol Clarity (14%), Liquidity Confirmation (10%), Regime Stability (8%) พร้อมหักคะแนนเมื่อ Macro ขัดแย้งกับ Trend หรือ Transition Risk สูง — ไม่ใช่แค่ค่า 0/1 อย่างระบบพื้นฐานทั่วไป

การป้องกันความผิดพลาดจากข้อมูลไม่พอ

เอนจินป้อนข้อมูล 250 แท่งให้ตัวจำแนกเสมอ (มากกว่า 200 แท่งที่ EMA200 ต้องการ) แต่ตัวจำแนกเองมีการป้องกันไว้ด้วย: หาก EMA200 ที่คำนวณ Fallback มาจากข้อมูลน้อยกว่า 200 แท่งจริง (เช่น กรณีถูกเรียกด้วยข้อมูลสั้นจากที่อื่น) ระบบจะไม่กล้าฟันธง Trend เป็น Bullish/Bearish และบังคับเป็น NEUTRAL พร้อมลด Confidence ลง — ป้องกัน "EMA200 ปลอม" จากข้อมูลไม่พอสร้างสัญญาณเทรดผิดพลาด

5.4 ตัวจับ Regime เชิงสถิติ (GMM Cross-check)

เพื่อยกระดับตัวจำแนกแบบ Rule-based (ที่ threshold ตั้งด้วยมือ) ระบบมีตัวจับ Regime แบบ **สถิติล้วน** (`xauby/regime/statistical.py`) ทำงานคู่ขนานเป็น "ความเห็นที่สอง" ที่ไม่มีอคติจาก Threshold ที่คนตั้ง:

- Fit **Gaussian Mixture Model (GMM)** แบบ **diagonal covariance** ด้วย **EM** บนพื้นที่ Feature 2 มิติ: Trend (ผลรวม Log-return แบบ Rolling) และ Log Realized Volatility
- เลือก 2 แกนนี้เพราะข้อมูลจริงพิสูจน์แล้วว่า Regime แยกด้วย Volatility ได้ชัดเจนกว่าทิศทาง
- คืนค่า Cluster ของแท่งล่าสุด พร้อม **Posterior Probability** และ **Persistence เชิงประจักษ์ (Stay Probability)**
- ใช้ **numpy pure** ไม่มี scipy/sklearn เพื่อให้ `classifier.py` หลักยังคง pure-stdlib

ผลลัพธ์แบบเป็น Feature `stat_*` เข้ากับ Regime ปกติ เป็น **Advisory เท่านั้น** — **ไม่เคย Override Regime หรือ Routing ใดๆ** ปิดอยู่โดย Default หลัง Flag `architecture.regime_statistical_crosscheck` ปรับค่าได้ผ่าน Block `regime_statistical: (components, minBars)`

5.5 รายงานความแม่นยำของ Regime Detector

"ความแม่นยำ" ของ Regime Detector **ไม่ใช่** "ทายถูกที่เปอร์เซ็นต์" เพราะไม่มี Ground Truth ว่าตลาด ณ ขณะนั้นอยู่ Regime ไหนจริงๆ ให้เทียบ การวัดที่ถูกต้องมี 3 อย่าง:

- 1. **Volatility Separation** — Regime แยกสภาพ Volatility จริงได้หรือไม่ (งานหลักที่ขับเคลื่อน Position Sizing และความกว้าง Stop)
- 2. **Persistence** — Regime อยู่นานพอให้ลงมือทำอะไรได้ หรือสลับไปมาทุกแห่ง (Churn)
- 3. **Directional Tilt** — Regime กลุ่ม Bull ให้ Drift สูงกว่ากลุ่ม Bear จริงหรือไม่ (คาดหวังว่าจะ**เส็ก**เสมอ เป็นโบนัส ไม่ใช่ตัวชี้วัดหลัก)

วัดด้วย **eta² (สัดส่วนความแปรปรวนของ Return ที่อธิบายได้ด้วย Regime Label)** พร้อม Permutation p-value (ความน่าจะเป็นที่การสุ่มจัดกลุ่มใหม่จะแยกข้อมูลได้ดีเท่ากัน) วัดจากข้อมูลจริง 4 ชั่วโมง (PAXGUSDТ ตัวแทนทองคำ ~12,800 แท่ง / BTCUSDТ ~17,500 แท่ง)

ตัวชี้วัด	PAXGUSDТ (ตัวแทนทองคำ)	BTCUSDТ (cross-check)
Volatility separation eta ² (p)	0.041 (p = 0.0005)	0.034 (p = 0.0005)
Directional separation eta ² (p)	0.0004 (p = 0.85 — เท่า Noise)	0.0019 (p = 0.0015)
Persistence (Mean run / Flip rate)	6.6 แท่ง / 15.1%	5.2 แท่ง / 19.2%
ลำดับ Drift (Bull เทียบ Bear)	+1.3 เทียบ -0.0 bps ✓	+5.8 เทียบ +0.5 bps ✓
GMM Cross-check Agreement	50.0%	45.2%

บทสรุป: ตัวจำแนกทำหน้าที่หลักได้ดี — แยก Volatility ได้ชัดเจนและมีนัยสำคัญทางสถิติจริง (p ≤ 0.001 ทั้งสองสินทรัพย์), Regime นิ่งพอจะเทรด (Mean run > 3 แท่ง แปลว่า Debounce 3 แท่งของระบบตั้งไว้เหมาะสม), และลำดับ Bull/Bear ถูกทิศทาง ส่วน Directional Edge อ่อนตามคาด (ปกติของ Classifier ทุกตัวในตลาดจริง) **ข้อสรุปเชิงปฏิบัติ: ใช้ Regime เพื่อปรับขนาดไม่ตาม Volatility ไม่ใช่ทำนายทิศทางแท่งถัดไป**

GMM Cross-check เห็นตรงกับ Rule-based เพียง ~45-50% ของเวลา ซึ่ง**ตั้งใจให้เป็นแบบนั้น** — ถ้า Cross-check ให้ผลตรงกันเกือบ 100% แปลว่ามันไม่ใช่ความเห็นที่สองจริงๆ การไม่ตรงกันคือสัญญาณว่าการอ่านตลาด ขณะนั้นกำกวม เป็นข้อมูลที่มีค่าสำหรับผู้ดูและระบบหรือ Confidence Gate ในอนาคต ไม่ใช่ข้อบ่งชี้ว่าตัวใดตัวหนึ่งผิด

ข้อจำกัดที่ควรรู้

- ค่า eta² เล็กในเชิงสัมบูรณ์ (~0.04) — ตลาดส่วนใหญ่คือ Noise ไม่มี Regime Model ใดอธิบายความแปรปรวนได้ มาก การที่ 0.04 มี p ≤ 0.001 แปลว่าการแยกนั้น "จริงและมั่นคง" ไม่ใช่ "ใหญ่"
- ประวัติราคาทองใช้ PAXGUSDТ เป็นตัวแทน (Proxy) เพราะ XAUUSDТ เองมีประวัติสั้นกว่า

- Threshold ยังตั้งด้วยมืออยู่ ยังไม่ได้ Fit ต่อสินทรัพย์ — GMM Cross-check คือก้าวแรกสู่โมเดลที่ขับเคลื่อน ด้วยข้อมูลมากขึ้น
- Forward Return เป็นสถิติเชิงพรรณนาแบบ In-sample ไม่ใช่ผล Backtest ที่พิสูจน์ว่ากลยุทธ์ทำกำไรได้

คำสั่งสร้างรายงานซ้ำ: `python3 scripts/regime_forward_validate.py --csv <candles.csv> --timeframe 4h --horizon 6`

5.6 RegimeRouter — เลือกกลยุทธ์ตาม Regime อัตโนมัติ

RegimeRouter คือระบบที่แมป Regime แต่ละแบบไปยังกลยุทธ์ที่เหมาะสม เป็น Opt-in ต่อคู่เทรด:

1. Flag Global `architecture.regime_router_enabled` ต้องเป็น true
2. Flag ต่อสินทรัพย์ `regime_router_enabled` ต้องเป็น true
3. ถ้าเป็นคู่ Live ต้องมี `regime_router_live_confirmed: true` เพิ่มด้วย

ถ้าสินทรัพย์ Live เปิด Router โดยไม่มี Live Confirmation เอนจินจะบังคับกลับเป็น Sim อัตโนมัติ พฤติกรรมอื่นๆ ของ Router: Debounce 3 แท่งก่อนยืนยันการสลับ Regime, NO_TRADE บล็อกการ BUY ใหม่แต่คง Stop Protection ของโพซิชันเดิม, Recovery ต้องผ่าน 3 แท่งที่เทรดได้ก่อนกลับสู่ปกติ, บังคับปิดหลัง 6 แท่ง NO_TRADE ติดต่อกัน, และ Handoff กลยุทธ์ยึดหลักความปลอดภัยตามหัวข้อ 3.2

สถานะจริงตอนนี้: คู่เทรด XAU ปิด RegimeRouter อยู่ (`regime_router_enabled: false`) — หมายความว่าตอนนี้ระบบรับ `cdc_action_zone` ตรงๆ ไม่ผ่านการเลือกกลยุทธ์อัตโนมัติ ตารางด้านล่างคือ Mapping ที่ Config ไว้ในระบบ (พร้อมใช้เมื่อเปิด Flag ในอนาคต):

Regime	กลยุทธ์ที่ Map ไว้	Action
BULL_BREAKOUT	<code>cdc_action_zone</code>	BUY ได้
BULL_TREND_STRONG	<code>cdc_action_zone</code>	BUY ได้
BULL_TREND_WEAK	<code>cdc_action_zone</code>	BUY ได้
LOW_VOL_ACCUMULATION	<code>bbrsi_mean_reversion</code>	BUY ได้
LOW_VOL_RANGE	<code>bbrsi_mean_reversion</code>	BUY ได้
VOLATILITY_EXPANSION	<code>supertrend_ema200</code>	BUY ได้
SIDEWAYS_CHOP	<code>bbrsi_mean_reversion</code>	BUY ได้
BEAR_TREND_WEAK	<code>cdc_action_zone</code>	BUY ได้
PANIC_SELL	None	NO_TRADE
BEAR_BREAKDOWN	None	NO_TRADE
BEAR_TREND_STRONG	None	NO_TRADE

เมื่อเปิด Regime Filter สำหรับ Backtest ระบบจะเลียนแบบ Routing นี้ด้วย: บล็อกสัญญาณ BUY ทุกครั้งที่ Regime อยู่นอกกลุ่มเป้าหมายของกลยุทธ์นั้น เพื่อให้ผล Replay ใกล้เคียงกับประตู่เข้าไม้ของ Live มากที่สุด

ความขัดแย้งในเอกสารต้นฉบับที่ควรทราบ: mapping ข้างบนอ้างอิงจากตารางใน root README.md แต่ docs/configuration.md ระบุ mapping ต่างออกไป สำหรับบาง Key เช่น BEAR_TREND_WEAK → donchian_trend (แทนที่จะเป็น cdc_action_zone) และกลุ่ม Low-vol → bbkc_squeeze (แทนที่จะเป็น bbrsi_mean_reversion) นอกจากนี้ root README ยังมีข้อความว่า donchian_trend "ผ่านการ deploy บน BTCUSDT 1h แล้ว" ในขณะที่ docs/regime_strategy_selection.md (มีระเบียบวิธีและวันที่ล่าสุดกว่า คือ 2026-06-11) สรุปตรงข้ามว่า **ไม่มี candidate ตัวไหนผ่านเกณฑ์ และ mapping เดิมยังคงอยู่ทั้งหมด** — นี่คือ doc drift จริงระหว่างเอกสารสองไฟล์ในโปรเจกต์เดียวกัน หนังสือเล่มนี้เลือกใช้ เวอร์ชันจาก docs/regime_strategy_selection.md เป็นหลักในหัวข้อ 5.7 เพราะมีข้อมูล/วิธีวัดผลละเอียดกว่าและวันที่ใหม่กว่า แต่ผู้ดูแลระบบควรตรวจสอบ regime_router.mapping จริงใน bot_config.yaml เสมอ แทนที่จะเชื่อเอกสารฉบับใดฉบับหนึ่งเพียงอย่างเดียว

5.7 กระบวนการคัดเลือกกลยุทธ์ด้วยข้อมูลจริง (R&D)

ระบบมี Gatekeeper แบบขับเคลื่อนด้วยข้อมูลสำหรับตัดสินใจเปลี่ยนกลยุทธ์ใน regime_router.mapping : กลยุทธ์ผู้ทำซิงจะแทนที่กลยุทธ์เดิมได้ **ก็ต่อเมื่อ** ชนะ Profit Factor ของกลยุทธ์เดิมทั้งชุด Train (70% เก่าสุด) และ Test (30% ใหม่สุด) ของข้อมูลตลาดจริงหลายปี พร้อมมีจำนวนไม้พ้อและ Drawdown อยู่ในขอบเขต — **ข้อมูลเป็นคนตัดสินใจ ไม่ใช่ความเชื่อ**

เกณฑ์ตัดสินใจ (ต้องผ่านครบทุกข้อ)

1. PF ผู้ทำซิง > PF เดิม ทั้ง Train และ Test
2. จำนวนไม้ ≥ 20 (Train) และ ≥ 8 (Test)
3. $PF_{test} \geq 1.0$ และ $maxDD_{test} \leq 1.5$ เท่าของเดิม (และ $\leq 15\%$ แบบสัมบูรณ์)
4. Window ล่าสุด ~6 เดือน: $PF \geq 0.8$ (Sanity check ว่าไม่พลิกหัวกลับในตลาดปัจจุบัน)

ไม่ผ่านข้อใดข้อหนึ่ง = กลยุทธ์เดิมอยู่ต่อสำหรับ Mapping Key นั้น

ผลการตัดสินใจล่าสุด (มี.ย. 2026, BTCUSDT 1h): จากข้อมูลจริง 26,297 แท่ง ไม่มี Candidate ตัวไหนผ่านเกณฑ์ครบ — donchian_trend ชนะ Train ชัดเจน (PF 1.16 เทียบ 0.77) แต่พียงบน Test (PF 0.32 ต่ำกว่าทั้งเกณฑ์ 1.0 และ PF เดิม) จึง **คงกลยุทธ์เดิมทั้งหมด** ไว้ในทุกกลุ่ม Regime กระบวนการนี้เป็นงานวิจัยบน BTCUSDT ซึ่งปัจจุบันไม่ใช่คู่เทรด Live ของระบบ — ใช้เป็นตัวอย่างระเบียบวิธีสำหรับ การตัดสินใจเปลี่ยนกลยุทธ์ในอนาคต

คำสั่งที่เกี่ยวข้อง: `scripts/fetch_global_klines.py`, `scripts/regime_strategy_eval.py --grid`, `scripts/regime_churn_analysis.py` (วัดความถี่การสลับ Regime และผลกระทบต่อการรัน กลยุทธ์จริง)

6.1 การคำนวณขนาดโพสิชัน (Position Sizing)

ขนาดไม้คำนวณแบบอิงความเสี่ยงและทบต้นอัตโนมัติ:

```
qty = (equity * risk_pct) / sl_distance
```

ถูกจำกัดเพดานด้วย `max_position_per_trade_pct` และ Allocation ต่อสินทรัพย์ (`allocation_pct`) Equity รวมโพสิชันที่เปิดอยู่ ณ ราคา Mark ด้วย ทำให้กำไรที่ทำได้ ไม่ขยายขนาดไม้ถัดไปโดยอัตโนมัติ (Auto-compounding)

ข้อสำคัญ: `risk_pct` ต้องตรงกันระหว่าง `trading.risk_pct` (Live อ่านจาก `portfolio.position_sizing.risk_pct`) และ `portfolio.position_sizing.risk_pct` (Backtest อ่านจาก `trading.risk_pct`) — ค่าไม่ตรงกันจะทำให้ Live กับ Backtest มีผลไม่ตรงกัน (parity พัง) ค่าปัจจุบันของระบบคือ `0.02` (2%) ทั้งสองจุด ระบบมี Startup guard (`MAX_SAME_RISK_PCT = 0.10`) ที่ปฏิเสธค่า `risk_pct` เกิน 10% กันที่

6.2 Stop-Loss / Trailing / Breakeven

กลไก Exit ที่ระบบรองรับ (ไม่ใช่ทุกตัวเปิดใช้ในทุกกลยุทธ์ — ดูค่าจริงของ CDC ในหัวข้อ 5.2):

- **Initial ATR Stop-Loss** — ระยะเริ่มต้นจาก Entry คูณด้วย `sl_atr_mult`
- **Trailing Stop** — ขยับตาม ATR คูณ `trailing_atr_mult` เมื่อราคาไปในทางที่ดี
- **Breakeven** — เลื่อน SL มาที่จุดเข้าเมื่อกำไรถึงเกณฑ์ที่ตั้ง (ถ้าเปิดใช้งาน)
- **Multi-tick Local SL Confirmation** — สำหรับสายที่ไม่มี SL ฝั่ง Exchange ต้องยืนยันคน `sl_confirm_ticks` ก่อนตัดสินใจปิดไม้จริง เพื่อกันสัญญาณหลอกจาก Spike ราคาสั้นๆ

6.3 Partial Take-Profit

Strategy Config เปิด Partial Take-Profit แบบ One-shot ได้:

```
strategy:
  params:
    cdc_action_zone:
      partial_tp_pct: 12.0
      partial_tp_fraction: 0.5
```

เมื่อโพสิชันที่เปิดอยู่ถึงผลตอบแทน `partial_tp_pct` จากราคาเข้า เอนจินจะปิด `partial_tp_fraction` ของส่วนที่เหลือด้วย Reduce-only Close บน Swap จริง (หรือผ่าน SimBroker ในโหมด Sim) บันทึกลงเป็น Partial Closed Trade, ตั้ง `trade_states.partial_tp_taken=1`, และปล่อยส่วนที่เหลือให้ออกตามกลยุทธ์ปกติ ทริกเกอร์นี้เป็นแบบ One-shot ต่อโพสิชัน และคงอยู่แม้ผ่านการ Restart สำหรับ Baseline ของค่า Live ปัจจุบัน นี่หมายถึงการเก็บ 50% ที่ +12% แล้วปล่อยส่วนที่เหลือขึ้นไปจนกว่า CDC จะออก หน้าจอ UI แสดงเป็น `PTP`, `Partial TP`, หรือ `PTP banked` เมื่อดำเนินการแล้ว

6.4 Drawdown Guard — สวิตช์ตัดขาดทุนระดับพอร์ต

`risk.drawdown_guard` คือ Kill-switch ระดับพอร์ตโดยรวม: บล็อกการ BUY ใหม่ (และบังคับปิดโพสิชันถ้าตั้ง `close_positions: true`) เมื่อ Equity ร่วงลงต่ำกว่า `max_drawdown_pct` จากจุดสูงสุดที่เคยบันทึกไว้ (High-water Mark ที่เก็บใน `core/equity_peak.json`)

6.5 การเทรด Short และเงื่อนไขความปลอดภัย

คู่เทรดปัจจุบันเปิด Short ได้จริงบน Live (`allowed_sides: [long, short]`, `short_live_enabled: true`) เพราะ Adapter ของ OKX Swap ประกาศความสามารถ `swap / positions / reduce_only` ครบ เงื่อนไขความปลอดภัยของ Short:

- ต้องอนุญาตต่อคู่เทรดอย่างชัดเจนผ่าน `allowed_sides`
- Live ต้องเปิด `short_live_enabled` เพิ่มเติม — ไม่ขึ้นเป็น Paper-only
- Startup ล้มเหลวทันที (Fail closed) ถ้า Adapter ที่เลือกไม่มีความสามารถ Swap/Positions/Reduce-only
- Leverage Default 1 เท่า และเพดานสูงสุด 3 เท่า
- ช่องทาง Streaming (CCXT Pro) ถ้าเสื่อมสภาพจะบล็อก Short ใหม่ทันที; REST Fallback จะไม่เปลี่ยน Exchange/Market Type

6.6 เช็กลิสต์ความสอดคล้องของค่าความเสี่ยง

ควรตรวจสอบให้ตรงกันก่อนเทรด Live:

Key	ค่าปัจจุบันของ XAU
<code>trading.risk_pct</code>	0.02
<code>portfolio.position_sizing.risk_pct</code>	0.02
<code>trading.max_position_per_trade_pct</code>	25.0
<code>trading.max_daily_loss_pct</code>	6.0
<code>trading.max_open_positions</code>	1
<code>derivatives.max_leverage</code>	1

Guard ระดับ Global ใช้กับทุกสินทรัพย์เสมอ ส่วนสถานะความเสี่ยงระดับ Strategy ต่อคู่เทรดถูกแยกจากกัน แต่เงินทุนพอร์ตยังใช้ร่วมกันโดยเจตนา

7.1 แหล่งข้อมูลสำหรับ Backtest

Backtest ดึงข้อมูลจาก **Global Binance** (`api.binance.com`) เป็น Default เพื่อประวัติราคาที่ยาวกว่า (`backtest.data_base_url`, override ผ่าน env `BINANCE_BACKTEST_URL`) ส่วนการเทรด Live ใช้ Endpoint OKX ที่ตั้งค่าไว้จริง Backtest และ Live resolve Config ผ่าน Resolver ตัวเดียวกันเสมอ เพื่อให้ Backtest, Replay, และ Live สอดคล้องกับแหล่งความจริงเดียวกัน

7.2 คำสั่ง Backtest และ Optimizer

```
python scripts/replay_backtest.py --symbol XAUUSD --config bot_config.yaml
python scripts/optimize_pair_configs.py
```

การเลือกกลยุทธ์เป็น Dry-run โดย Default:

```
python scripts/select_pair_strategy.py --symbol BTCUSD --candidates a,b,c
python scripts/select_pair_strategy.py --symbol BTCUSD --candidates a,b,c --apply
```

`--apply` เท่านั้นที่จะเขียนผู้ชนะที่ผ่านเกณฑ์กลับเข้า YAML

7.3 Regime-aware Replay

การตั้ง `use_regime_filter: true` บน Strategy Config ของสินทรัพย์จะเปิด Regime Classifier ภายใน Replay Engine — สัญญาณ BUY จะถูกระงับนอกกลุ่ม `TARGET_REGIMES` ของกลยุทธ์นั้น และประเมินใหม่ทุก `regime_updateBars` แท่ง (Default 24 แท่ง)

7.4 Replay Validation หลัง Restart/Incident

ใช้หลังการ Restart, Incident, หรือเปลี่ยน Config เสมอ:

```
python scripts/replay_validate.py <run_id> --symbol XAUUSD
```

Replay Validation ตรวจสอบว่า Event `signal_evaluated` ที่บันทึกไว้จาก Live ตรงกับผล Replay ของกลยุทธ์เดียวกันหรือไม่ — ตรวจสอบเฉพาะผลลัพธ์ของกลยุทธ์ ก่อนจะถูก Override โดย Macro Guard, Cooldown, และ Execution

8.1 Textual TUI (แดชบอร์ดในเทอร์มินัล)

xAuby มี UI แบบ Textual ในเทอร์มินัลที่อ่าน State ของเอนจินแบบ Real-time — **ไม่ได้รับลูปเทรดเอง และเปิด Database แบบ Read-only เสมอ**

หน้าจอ	คีย์	หน้าที่
Dashboard	1 , d	กราฟ, ตารางคู่เทรด, Regime, โพชิชัน, บัญชีกลยุทธ์/โหมด
Trade log	2 , t	ไม้ที่เปิดแล้วและประวัติ
Incident explorer	4 , i	Timeline การรันจาก Event store
Menu	Launcher/env	Action ด้านสำหรับเอนจิน, Dashboard, เทสต์, ตัวช่วย Config

คีย์ Global: **[/ 0** คู่เทรดก่อนหน้า, **] / 9** คู่เทรดถัดไป, **q** ออก

กราฟและ Legend เดินตามกลยุทธ์ที่ Active ของคู่เทรดที่เลือกเสมอ (Source of Truth ตามหัวข้อ 3.5) UI ปรับตามความกว้างเทอร์มินัล: ≥ 110 คอลัมน์ = สองคอลัมน์ (กราฟซ้าย สถิติขวา), $75-109$ = คอลัมน์เดียวเต็มความกว้าง, < 75 = แถวแบบมือถือ กราฟสั้นลง

เมื่อมีโพชิชันเปิดและ Config มี `partial_tp_pct` Panel โพชิชันจะแสดงแถว PTP พร้อมสัดส่วนที่จะเก็บ, ราคาทริกเกอร์, และสถานะ (`pending` จนกว่าจะปิดสำเร็จ แล้วเปลี่ยน เป็น `banked`)

รันผ่าน tmux (แนะนำบน VPS):

```
./scripts/start_dashboard_tmux.sh
./scripts/attach_dashboard_tmux.sh
```

8.2 WebUI มือถือ (Read-only)

WebUI คือแดชบอร์ดเบราว์เซอร์ขนาดเล็กแบบอ่านอย่างเดียว สำหรับเช็คบอกจากเดสก์ท็อปหรือมือถือ — **ไม่วาง ไม่ยกเล็ก ไม่หยุด และไม่กลับมาเทรดใดๆ ทั้งสิ้น**

```
./scripts/start_webui.sh
```

Bind ค่าเริ่มต้น: `127.0.0.1:8787`

หน้า Home แสดงสถานะ Runtime ของ OKX สด, แท่งเทียน OHLC 24 แท่งล่าสุด, เส้น EMA12/EMA26 ซ้อนทับ, รายละเอียด CDC Action Zone, และสถานะ Partial TP ของโพชิชันที่เปิดอยู่ หน้า Activity มีสโครลของตัวเองสำหรับ Event และไม้ที่เปิดแล้ว

เข้าถึงจาก Windows

```
ssh -L 8787:127.0.0.1:8787 user@your-vps-ip
```

แล้วเปิด `http://localhost:8787`

เข้าถึงจากมือถือ

ตัวเลือกฟรีที่ง่ายที่สุดคือ Tailscale: ติดตั้งบน VPS และมือถือ, Login Tailnet เดียวกัน, ใช้ Tailscale Serve หรือ Bind WebUI ไปที่ IP Tailscale ของ VPS:

```
WEBUI_HOST=<vps-tailscale-ip> WEBUI_PORT=8787 ./scripts/start_webui.sh
```

ห้าม Bind WebUI ไปที่ `0.0.0.0` บน VPS สาธารณะ เว้นแต่จะเพิ่มระบบยืนยันตัวตนที่เหมาะสม

Endpoint ทั้งหมดเป็น Read-only: `/api/state`, `/api/health`, `/api/recent-events`, `/api/trades`, `/api/candles`

8.3 ปรัชญาการออกแบบหน้าจอ

WebUI ออกแบบให้เหมือน "โต๊ะทองส่วนตัว" — ผู้ใช้คือเจ้าของบอกเพียงคนเดียว เชื้จาก iPhone (Safari, มักผ่าน Tailscale) หรือเดสก์ท็อป บริบทคือการ "เหลือบดู" ไม่ใช่นั่งทำงาน — คำถามเดียวที่ต้องตอบใน 10 วินาที: "บอกรยังทำงาน อยู่ไหม, Equity เท่าไหร่, ตอนนี้อยู่โซนอะไร, มีอะไรเกิดขึ้นไหม"

หลักการออกแบบ

- **Glanceable ก่อนเสมอ** — ลำดับชั้น: Equity → โซน → สัญญาณ → อื่นๆ ไม่ต้องเลื่อนจอ
- **สื่อความหมายเท่านั้น** — เขียว/แดง/ส้ม = กำไร/ขาดทุน/เตือน เท่านั้น; สีทอง (Brand) ใช้บอกตัวตน/การเลือกเท่านั้น ไม่เคยบอกสถานะ
- **Read-only แปลว่าสงบ** — ไม่มีองค์ประกอบใดที่บ่งบอกว่า "ควบคุมได้" ผิดปกติจะดังชัด ปกติจะเงียบ
- **มือถือมาก่อน, Safari มาก่อน** — ออกแบบตามจริงของ iOS Safari (Toolbar แบบ Dynamic, Safe Area, ปุ่มสัมผัส $\geq 44px$) เดสก์ท็อปได้ Layout มือถือแบบวางกลาง ไม่ใช่ Layout กว้างแยกต่างหาก
- **ตัวเลขคือ Interface** — ใช้ Tabular figures, ความละเอียดสม่ำเสมอ, คอลัมน์จัดตรงกัน Typography ทำหน้าที่แทนการตกแต่ง

โทนาลี (OKLCH)

Token	บทบาท
<code>--bg-0</code> / <code>--surface</code>	พื้นหลังหน้า / การ์ด-แพน (Dark only — <code>color-scheme: dark</code>)
<code>--gold</code>	Accent เดียวของแบรนด์ — ใช้บอกตัวตน/การเลือกเท่านั้น
<code>--pos</code> / <code>--neg</code> / <code>--warn</code>	กำไร/Long — ขาดทุน/Short — ระวัง/เตือนสภาพ
<code>--info</code>	ข้อมูลประกอบ เช่น EMA26, Regime info

Typography ใช้ Font Stack ระบบของ iOS เดียว (ไม่มี Webfont), ตัวเลขทั้งหมดใช้ `font-variant-numeric: tabular-nums` Layout แบบ Phone-first ความกว้าง `min(430px, 100%)` , สามมุมมอง (Home/Signal/Activity) สลับด้วย `body[data-view]`

9.1 การติดตั้ง Telegram Bot

1. สร้างบอทผ่าน [@BotFather](#)
2. หา Chat ID ของตัวเอง เช่น ผ่าน [@userinfobot](#)
3. ตั้งค่า `.env`:

```
TELEGRAM_ENABLED=true  
TELEGRAM_BOT_TOKEN=...  
TELEGRAM_CHAT_ID=...
```

เปิด Polling ใน `bot_config.yaml`:

```
notifications:  
  alert_channel: telegram  
  telegram_command_polling_enabled: true
```

9.2 คำสั่งทั้งหมด

อนุญาตเฉพาะ `TELEGRAM_CHAT_ID` ที่ตั้งค่าไว้เท่านั้น:

คำสั่ง	คำอธิบาย
<code>/help</code> , <code>/start</code>	รายการคำสั่ง
<code>/status</code>	ทุกคู่เทรด: ราคา, โหมด, โพซิชัน, สัญญาณล่าสุด
<code>/pnl</code>	กำไรขาดทุนพอร์ต 7 วัน และ 30 วัน พร้อมแยกต่อคู่เทรด
<code>/regime</code>	Regime snapshot ต่อคู่เทรด
<code>/last</code>	ไม้ที่ปิดล่าสุดต่อคู่เทรด
<code>/health</code>	Snapshot สุขภาพเอนจิน: โหมด, สถานะ Pause, คู่เทรด, โพซิชันเปิด, ปัญหา Feed/WS
<code>/pause</code>	ยืนยัน Pause ดูกเงิน — บล็อกการ BUY ใหม่โดยไม่ปิดโพซิชัน
<code>/resume</code>	ยืนยัน Resume — อนุญาต BUY ใหม่อีกครั้ง

9.3 การแจ้งเตือนอัตโนมัติ

เหตุการณ์	ระดับ
เริ่ม/หยุดเอนจิน	trade / info
BUY/SELL Fill	trade
Macro Guard บล็อก BUY	info
Regime Score/Label เปลี่ยน	info
RegimeRouter Handoff / NO_TRADE	info / position
Trailing Stop อัปเดต	position
กู้คืน SL ล้มเหลว	position / critical
Daily digest / Weekly review	info

9.4 โหมด Semi-auto และ Emergency Pause

```
trading:
  mode: semi_auto
notifications:
  semi_auto_confirm_timeout_seconds: 60
```

บอกระบบ Inline Keyboard (**Confirm BUY** / **Skip**) ก่อนเปิดไม้ทุกครั้ง

/pause และ **/resume** เป็นคำสั่งแบบ 2 ขั้นตอน: พิมพ์คำสั่งจะแค่ ถ้ามียืนยัน ปุ่ม Inline ต่างหากที่จะใช้ Control จริง Pause จะบล็อกการ BUY ใหม่ทุกเส้นทาง (สัญญาณ, Manual, Semi-auto) โดยไม่ปิดโพซิชันที่เปิดอยู่ Critical Alert มีปุ่ม **Status** และ **Ack** มาด้วยเสมอ

10.1 รับหลายอินสแตนซ์บนเครื่องเดียว

xAuby แยก Config ออกจาก Mutable Data ทำให้รับหลายเอนจิน (ต่างบัญชี/Exchange/ชุดคู่เทรด) พร้อมกันบนโค้ด Checkout เดียวกันได้โดยไม่ชนกัน:

ส่วน	ตัวแปร Env	ค่าเริ่มต้น
Config (<code>bot_config.yaml</code> , whitelist, <code>.env</code>)	<code>XAUBY_CONFIG_DIR</code>	โฟลเดอร์ทำงานปัจจุบัน
Mutable data (DB, lock, state, log, ยอด sim)	<code>XAUBY_HOME</code> + <code>XAUBY_INSTANCE_ID</code>	<code>core/</code> (+ <code>/<id></code>)

```
XAUBY_CONFIG_DIR=instances/acct1 XAUBY_INSTANCE_ID=acct1 \
python run_xauby.py --live
```

ตัวอย่าง systemd template สำหรับหลาย instance:

```
[Service]
Environment=XAUBY_CONFIG_DIR=/root/xAuby/instances/%i
Environment=XAUBY_INSTANCE_ID=%i
WorkingDirectory=/root/xAuby
ExecStart=/root/xAuby/venv/bin/python run_xauby.py --live
```

เปิดใช้ด้วย `systemctl enable --now xauby@acct1` — แต่ละ instance อ่าน `.env` ของตัวเอง ทำให้ Secret ไม่ต้องแชร์ข้ามบัญชี

10.2 Multi-Exchange ผ่าน CCXT

สลับ Exchange ได้ทั้งหมดผ่านเมนู Launcher โดยไม่ต้องแก้ไฟล์: option 4 → Exchange & Flags → Wizard จะเลือก Provider → `ccxt_id` → `quote_asset` → ใส่ Credential (Wizard เขียนตัวแปร `.env` ที่ถูกต้องให้เอง) → รับ Connection Test → เสร็จ Restart

```
exchange:
  provider: ccxt
  ccxt_id: kraken
  api_key_env: KRAKEN_API_KEY
  api_secret_env: KRAKEN_API_SECRET
  quote_asset: USDT
```

Exchange Plugin Registry + WebSocket Streaming

เปิดใช้ด้วย Flag:

```
architecture:
  exchange_plugin_registry_enabled: true
```

เมื่อเปิด: `provider: binance` → Client แบบ Native + BinanceWebSocket, `provider: ccxt` (หรือ `ccxt_id` ใดก็ได้)
→ CCXTExchangeClient + CCXTProWebSocket (Streaming จริงสำหรับทุก Exchange ที่ `ccxt.pro` รองรับ)

เพิ่ม Exchange ใหม่ (Drop-in ไม่ต้องแก้ Factory)

สร้างไฟล์เดียว `xauby/api/exchanges/exchange_<name>.py` ที่ลงทะเบียนทั้ง REST Gateway และ WebSocket Adapter ผ่าน Decorator `@register_exchange` และ `@register_ws` — ระบบค้นพบไฟล์อัตโนมัติผ่าน `pkgutil` (Filename prefix `exchange_`)

10.3 OKX Perpetual และความปลอดภัยของ Short

เส้นทาง Derivatives ที่รับรองแล้วใช้ `market_type: swap`, `margin_mode: isolated`, `position_mode: one_way`, และ Settle Asset เป็น USDT สัญลักษณ์ในเอนจินยังคงกระชับ (`BTCUSDT`) ขณะที่ Adapter resolve เป็นสัญญา Native (`BTC/USDT:USDT`) และแปลงจำนวนฐานเป็น Contract ด้วย `contractSize`

Startup จะล้มเหลวทันที (Fail closed) เมื่อ Adapter ที่เลือกไม่มีความสามารถ Swap, Positions, และ Reduce-only อนุ CCXT Pro รับ Subscribe Ticker, OHLCV, Public Trades, และ Order Book ทุก Market Event มี Identity ของ Exchange/Market กำกับ ช่องทางที่เสื่อมสภาพจะบล็อก Short ใหม่ทันที ส่วน REST Fallback จะไม่เปลี่ยน Exchange หรือ Market Type เด็ดขาด

11.1 การ Deploy บน VPS

```
mkdir -p core/logs
env -u DEFAULT_SYMBOL XAUBY_DEFAULT_SYMBOL=XAUUSDT XAUBY_FOCUS_SYMBOL=XAUUSDT \
SIMULATE_ONLY=false BOT_READ_ONLY=false \
./venv/bin/python run_xauby.py --live --pair XAUUSDT

./scripts/start_dashboard_tmux.sh
./scripts/attach_dashboard_tmux.sh
```

ใช้เงินจริงเพียง 1 Instance ต่อ 1 Runtime Root เท่านั้น `<root>/engine.lock` ป้องกันการซ้ำซ้อนภายใน Root เดียวกัน

11.2 คำสั่งอัปเดตและรีสตาร์ท

สำหรับปฏิบัติการปกติบน VPS หลังติดตั้ง CLI แล้ว ให้ใช้:

```
xauby update # deploy origin/main + controlled restart
xauby restart # controlled restart โดยไม่ต้องไดคัทใหม่
```

Fallback แบบ Manual:

```
scripts/deploy_from_github.sh --restart --branch=main
scripts/controlled_restart_engine.sh
```

11.3 การตรวจสอบ Latency ของ VPS

ใช้เมื่อต้องการเช็คช้า, TUI ค้าง, หรือ Request Exchange คมดเวลา:

```
./venv/bin/python scripts/vps_latency_check.py
./venv/bin/python scripts/vps_latency_check.py --json
```

คำที่ดู	ความหมาย
curl total	เวลารวมของ Request HTTPS ไปยัง Exchange API
curl ttfb	เวลาถึง Byte แรก — สูงมักแปลว่า Routing/API ค้าง
tcp443	เวลา Connect TCP ดิบ — สูงชี้ไปที่ระยะทาง Network/Routing
clock	<code>NTPSynchronized=yes</code> สำคัญเพราะ Request ที่เขียนลายเซ็นใช้ Timestamp
loadavg	Load สูงต่อเนื่องเกิน CPU ที่มีทำให้จอ/TUI ค้าง
disk_used/free	พื้นที่ต่ำหรือ Disk เข้ากระหนการเขียน SQLite/Event/State

TUI Header แสดง Metric: **REST** (Latency Request ล่าสุด), **WS** (อายุ Tick WebSocket ล่าสุด), **Tick** (ระยะเวลา Tick ทั้งหมด), **Sync** (ระยะเวลา Sync แท่งเทียน), **State** (ระยะเวลา Export State JSON) — ถ้า REST สูงแต่ Tick ต่ำ ปัญหาอาจจะอยู่ที่เส้นทาง VPS/Exchange ถ้า Tick/Sync/State สูง ให้ปรับงาน Local หรือลดความถี่ Poll/Log

11.4 เช็กลิสต์ความปลอดภัยฉบับเต็ม

- ☐ รีวิว Simulation Run แล้วก่อน Live
- ☐ API Key OKX มีสิทธิ์เทรดสำหรับบัญชีที่ตั้งใจ และ**ไม่มี**สิทธิ์ Withdraw
- ☐ ตัวแปร OKX API Key/Secret/Passphrase ครบก่อน Restart แบบ Live
- ☐ เงินอยู่ในบัญชี OKX ที่บอกอ่านสำหรับเทรด USDT Swap; Dashboard อาจแสดง 0.00 ถ้าเงินอยู่ผิดกระเป๋า
- ☐ นกทวน **risk_pct**, **max_position_per_trade_pct**, และ Guard ระดับ Global ให้เหมาะกับขนาดบัญชี
- ☐ ตั้งเกณฑ์ **risk.drawdown_guard** ให้ตรงกับความเสี่ยงที่รับได้ (Kill-switch)
- ☐ นกทวนผลประเมินการย้าย Long/Short ของ OKX XAU หลังดาวนโหลดแท่งเทียน 4H และ Funding History ใหม่
- ☐ **regime_router_live_confirmed: true** ตั้งเฉพาะหลังอนุมัติต่อคู่เทรดโดยตรง
- ☐ **architecture.strategy_sandbox_strict: true** เมื่อรันปลั๊กอินจากภายนอก/ไม่ไว้ใจ
- ☐ รีวิวไฟล์ **scripts/select_pair_strategy.py** ก่อนใช้ **--apply**
- ☐ **scripts/replay_validate.py <run_id> --symbol <PAIR>** ผ่านหลัง Restart
- ☐ ทดสอบ Telegram แล้ว: **/status** แสดงครบทุกคู่เทรด, **/health** ไม่มีปัญหาที่ไม่คาดคิด
- ☐ ทดสอบปุ่มยืนยัน Emergency **/pause** และ **/resume** ในโหมด Simulation แล้ว
- ☐ เอนจินถูกดูแลผ่าน tmux/systemd ไม่ใช่ SSH shell ที่ไม่มีคนดูแล

ก. อภิธานศัพท์ (Glossary)

คำศัพท์	ความหมาย
ATR	Average True Range — ค่าเฉลี่ยความผันผวนของราคาต่อแท่ง ใช้กำหนดระยะ SL/Trailing
EMA	Exponential Moving Average — ค่าเฉลี่ยเคลื่อนที่แบบถ่วงน้ำหนักเอนเอียงไปทางข้อมูลล่าสุด
PF (Profit Factor)	อัตราส่วนกำไรรวมต่อขาดทุนรวม — มากกว่า 1 แปลว่าโดยรวมเป็นกำไร
DD (Drawdown)	ระยะที่ Equity ร่วงลงจากจุดสูงสุดที่เคยทำได้
Regime	สภาพตลาดที่ถูกจัดหมวดโดยตัวจำแนก เช่น BULL_TREND_STRONG, PANIC_SELL
Debounce	การรอให้สัญญาณซ้ำกันครบจำนวนแท่งก่อนยืนยันการเปลี่ยนแปลงจริง (กัน Noise)
NO_TRADE	Regime ที่ระบบตัดสินใจไม่เปิดไม้ใหม่ แต่ยังบริหารไม้เดิมต่อ
Handoff	การส่งมอบการบริหารโพซิชั่นจากกลยุทธ์เดิมไปกลยุทธ์ใหม่อย่างปลอดภัย
Partial TP	การปิดโพซิชั่นบางส่วนที่กำไรถึงเกณฑ์ที่ตั้งไว้ (One-shot)
Reduce-only	ออเดอร์ที่รับประกันว่าจะลดขนาดโพซิชั่นเท่านั้น ไม่เปิดโพซิชั่นใหม่
Sandbox Strategy	การสแกน AST แบบ Static เพื่อปฏิเสธปลั๊กอินที่แตะ Engine/DB/Network หรือใช้ eval/exec
GMM (Gaussian Mixture Model)	โมเดลสถิติที่จัดกลุ่มข้อมูลด้วยการผสมของ Normal Distribution หลายตัว
eta ² (Eta-squared)	สัดส่วนความแปรปรวนของข้อมูลที่อธิบายได้ด้วยตัวแปรจัดกลุ่ม (ในที่นี้คือ Regime Label)
Permutation p-value	ความน่าจะเป็นที่การสุ่มจัดกลุ่มข้อมูลใหม่จะให้ผลแยกได้ดีเท่ากับของจริง — ค่า = บัญชีสำคัญสูง

ข. โครงสร้างโปรเจกต์

```
xAuby/  
|-- run_xauby.py  
|-- launcher.py           # thin shim re-exporting the xauby.launcher package  
|-- bot_config.yaml  
|-- coin_whitelist.json  
|-- docs/  
|-- scripts/  
|-- tests/  
`-- xauby/  
    |-- launcher/         # interactive launcher: config_io, process, maintenance, quick_config  
    |-- engine/           # LiteTradingEngine (mixins), risk, orders, regime routing  
    |-- strategies/       # strategy plugins + indicators/  
    |-- runtime/          # pair registry, whitelist validation, config resolvers  
    |-- regime/           # regime classifier + GMM cross-check  
    |-- backtest/         # replay engine, optimizer, metrics  
    |-- observability/    # EventEmitter, JSONL store, replay validation  
    `-- ui/               # terminal chart + Textual TUI, dashboard, tradelog
```

ค. ตารางสคริปต์อ้างอิงทั้งหมด

คำสั่ง	หน้าที่
<code>python -m unittest discover -s tests -q</code>	รันชุดเทสต์ทั้งหมด
<code>python health_check.py</code>	ตรวจสอบสุขภาพระบบครั้งเดียว
<code>python scripts/replay_backtest.py --symbol XAUUSD --config bot_config.yaml</code>	รันทดสอบย้อนหลัง (Backtest)
<code>python scripts/incident_explorer.py list</code>	ดูรายการ Incident
<code>python scripts/replay_validate.py <run_id> --symbol XAUUSD</code>	ตรวจสอบ Replay หลัง Restart
<code>python scripts/evaluate_okx_xau_migration.py</code>	ประเมินการย้ายมาใช้ OKX XAU
<code>python scripts/select_pair_strategy.py</code>	คัดเลือกกลยุทธ์ (Dry-run/--apply)
<code>python scripts/optimize_pair_configs.py</code>	Optimizer แบบ Low-CPU
<code>python scripts/capture_tui_screenshots.py</code>	สร้างภาพหน้าจอ TUI ใหม่
<code>python scripts/fetch_global_klines.py</code>	ดึงแท่งเทียนหลายปีจาก Global Binance
<code>python scripts/regime_strategy_eval.py --grid</code>	ประเมินกลยุทธ์ต่อ Regime แบบ Gatekeeper
<code>python scripts/regime_forward_validate.py</code>	วัดความแม่นยำ Regime Detector บน ข้อมูลจริง
<code>python scripts/regime_churn_analysis.py</code>	วัดความถี่การสลับ Regime
<code>python scripts/vps_latency_check.py</code>	ตรวจ Latency ของ VPS

ง. คำเตือนความเสี่ยง (Disclaimer)

ซอฟต์แวร์นี้จัดทำขึ้นเพื่อการศึกษาและวิจัยเท่านั้น การเทรด Cryptocurrency และอนุพันธ์มีความเสี่ยงสูงต่อการสูญเสียเงินทุน ผลการดำเนินงานในอดีตไม่ได้เป็นเครื่องยืนยันผลลัพธ์ในอนาคต ผู้ใช้ต้องปฏิบัติตามข้อกำหนดการใช้งาน API ของ Exchange ที่เลือกใช้และกฎหมาย/ระเบียบข้อบังคับที่เกี่ยวข้องในเขตอำนาจของตนเองเสมอ ผู้จัดทำเอกสารนี้ไม่รับผิดชอบต่อความเสียหายใดๆ ที่เกิดจากการใช้งานซอฟต์แวร์นี้กับเงินทุนจริง

— จบเล่ม —